INTER RAPIDÍSIMO S.A.

DIRECCIÓN DE TECNOLOGÍA & ARQUITECTURA DE SOFTWARE

DOSSIER TÉCNICO INTEGRAL · ARQUITECTURA & LIDERAZGO DE SOFTWARE

Expediente de Arquitectura y Solución Técnica Integral

Consolidación de Arquitectura Hexagonal .NET 8, Frontend Angular 21 Signals, Modelo Relacional SQL Server 2022+, Infraestructura Cloud AWS con Terraform y Respuestas de Liderazgo Técnico

ESTADO: DOCUMENTACIÓN OFICIAL VERIFICADA (PRODUCCIÓN 100%)

CANDIDATO / ROL EVALUADO
Líder Técnico de Desarrollo & Cloud Architect

VERSIÓN DE ENTREGA
Build v1.0.0 Estable (Producción)

STACK TECNOLÓGICO
.NET 8 · Angular 21 Signals · SQL Server · Terraform · AWS

FECHA DE CONSOLIDACIÓN
Octubre 2026

Índice General del Dossier

01 Constitución y Principios Rectores del Proyecto
Gobernanza Técnica

02 Solución Técnica Integral: Prueba para Líder Técnico
Liderazgo & Diagnóstico ZYX

03 Arquitectura del Backend: Hexagonal, Mediador Nativo & CQRS (.NET 8)
Arquitectura de Software Backend

04 Arquitectura del Frontend: Zoneless, Signals & Standalone (Angular 21)
Arquitectura de Software Frontend

05 Arquitectura de Base de Datos: Modelo Relacional & Persistencia (SQL Server)
Modelo de Persistencia

06 Arquitectura Cloud en AWS con Terraform: IaC Modular & Multi-AZ
Infraestructura Cloud

01 | Constitución y Principios Rectores del Proyecto

GOBERNANZA TÉCNICA

Constitución del Proyecto

1. Stack: ASP.NET Core, Angular Signals y SQL Server; mediador nativo interno y prohibido el paquete externo MediatR.
2. Trazabilidad: Todo endpoint y caso de uso implementado traza a un requisito funcional de una spec aprobada.
3. Arquitectura: Dominio hexagonal con Specification Pattern aislado sin dependencias hacia infraestructura ni API.
4. Tests: Pruebas unitarias obligatorias con xUnit para especificaciones y handlers con cobertura mínima del 85%.
5. Persistencia: Acceso a SQL Server mediante adaptadores de repositorio con auditoría canónica y soft-delete.
6. Idioma: Identificadores y código en inglés; mensajes de usuario y documentación funcional en español.

02 | Solución Técnica Integral: Prueba para Líder Técnico

LIDERAZGO & DIAGNÓSTICO ZYX

Solución Técnica Integral: Prueba para Líder Técnico

Empresa: INTER RAPIDÍSIMO S.A.
Candidato: Líder Técnico de Desarrollo y Arquitectura Cloud
Fecha: Octubre 2026

PARTE 1: Respuestas a Preguntas de Conceptos Clave (Nivel Líder Técnico)

1. ¿Cómo optimizarías una consulta lenta en SQL Server?

Rta//. Aplicaría un proceso metódico de diagnóstico y afinamiento en 5 etapas:

1. **Análisis de Métricas y Planes de Ejecución:** Extraer el *Actual Execution Plan* mediante SQL Server Management Studio (SSMS) o Extended Events, analizando advertencias de costos altos (operadores *Table Scan*, *Index Scan*, *Hash Match* o derrames en `TempDB`).
2. **Revisión de Indexación:** Verificar si faltan índices sugeridos (*Missing Index Details*) o si los existentes sufren de fragmentación elevada (> 30%) o estadísticas desactualizadas (`UPDATE STATISTICS`), forzando estimaciones erróneas de cardinalidad.
3. **Optimización Sintáctica T-SQL (SARGability):**
   - Eliminar funciones sobre columnas en cláusulas `WHERE` (ej. cambiar `WHERE YEAR(Fecha) = 2026` por rangos `WHERE Fecha >= '2026-01-01' AND Fecha < '2027-01-01'`).
   - Reemplazar subconsultas correlacionadas o cursores fila por fila por operaciones basadas en conjuntos (`JOIN`, `APPLY` o Window Functions).
   - Evitar `SELECT *` y proyectar únicamente las columnas estrictamente requeridas para permitir *Covering Indexes*.
4. **Paralelismo y Concurrencia:** Evaluar el `CXPACKET` wait type para detectar skew de CPU, ajustando el *Cost Threshold for Parallelism* o utilizando el hint `OPTION (RECOMPILE)` en consultas parametrizadas con problemas de *Parameter Sniffing*.
5. **Aislamiento y Bloqueos:** Implementar `READ COMMITTED SNAPSHOT ISOLATION` (RCSI) a nivel de base de datos para evitar que las lecturas bloqueen escrituras y viceversa.

---

2. ¿Cuál es la diferencia entre un índice clustered y un índice non-clustered?

Rta//.

- **Índice Clustered (Agrupado):**
  - **Estructura física:** Define el **orden físico real** en el que se almacenan las filas de datos en las páginas de disco. El nivel hoja (*leaf level*) del árbol B-Tree **es la tabla misma**.
  - **Límite:** Solo puede existir **uno por tabla**, habitualmente asociado a la Llave Primaria.
  - **Uso ideal:** Consultas de búsqueda por rangos (`BETWEEN`, `>`, `<`), ordenamientos naturales (`ORDER BY`) y búsquedas de alta frecuencia por ID.

- **Índice Non-Clustered (No Agrupado):**
  - **Estructura física:** Estructura independiente y separada físicamente de la tabla. El nivel hoja contiene las columnas indexadas y un **puntero** (*Row Locator*) que señala a la fila real (el *Clustered Key* si la tabla está agrupada, o un RID si es un Heap).
  - **Límite:** Hasta 999 por tabla en SQL Server.
  - **Uso ideal:** Búsquedas por igualdad en claves secundarias (ej. `Email`, `DocumentoIdentidad`, `NumeroGuia`), filtros frecuentes en `WHERE` y como *Covering Indexes* empleando la cláusula `INCLUDE(...)` para resolver la consulta completa sin requerir un costoso *Key Lookup*.

---

3. Explica qué servicios tecnológicos utilizarías o cómo implementarías una arquitectura escalable en AWS para una aplicación .NET Core

Rta//. Diseñaría una arquitectura en 3 capas desacopladas, elástica y orientada a eventos:

- **Capa Edge y Distribución:**
  - **Amazon Route 53** con políticas de enrutamiento por latencia/salud.
  - **AWS WAF** integrado a **Amazon CloudFront** para filtrado de ataques L7 (SQLi, XSS, rate limiting) y CDN para assets estáticos.
- **Capa de Cómputo y Orquestación:**
  - **Application Load Balancer (ALB)** en subredes públicas para distribución HTTPS y SSL Offloading.
  - **AWS ECS Fargate** (Serverless Containers) alojando la API .NET Core en subredes privadas. Permite autoescalado horizontal (Target Tracking basado en CPU o peticiones por segundo) sin gestionar instancias de SO.
- **Capa de Datos y Almacenamiento:**
  - **Amazon RDS for SQL Server** en configuración Multi-AZ desplegado en subredes aisladas.
  - **Amazon ElastiCache (Redis)** como caché distribuida L2 para reducir la carga de lectura en la base de datos hasta en un 80% (patrón *Cache-Aside*).
- **Desacoplamiento Asíncrono:**
  - **Amazon SQS / EventBridge** para procesar tareas pesadas (generación de facturas, sincronización de guías logísticas) mediante microservicios worker en background.
- **Gestión de Configuración y Seguridad:**
  - **AWS Secrets Manager** para credenciales dinámicas y rotación de contraseñas.
  - **Amazon CloudWatch + AWS X-Ray** para observabilidad, métricas y trazabilidad distribuida.

---

4. ¿Cuál es la diferencia entre EC2 y Lambda en AWS y en qué casos usarías cada uno?

Rta//.

Criterio	Amazon EC2	AWS Lambda
Modelo	IaaS (Infraestructura como Servicio) - Servidor virtual completo.	Serverless / FaaS (Función como Servicio).
Gestión	El cliente administra el SO, parches, seguridad de red y runtime.	AWS administra el runtime, escalado, alta disponibilidad e infraestructura.
Escalabilidad	Basada en Auto Scaling Groups (minutos para inicializar instancias).	Instantánea y automática por cada evento (milisegundos).
Tiempo de Ejecución	Continuo e ilimitado (24/7).	Límite máximo de 15 minutos por invocación.
Costos	Pago continuo por instancia provisionada, esté activa o inactiva.	Pago por milisegundo de ejecución y volumen de invocaciones.

- **Cuándo usar EC2:** Aplicaciones monolíticas legadas con dependencias a nivel de sistema operativo (servicios de Windows, drivers ODBC específicos), procesos batch que superan los 15 minutos de ejecución o cargas predecibles de alto tráfico 24/7 donde instancias reservadas resultan más económicas.
- **Cuándo usar Lambda:** Procesamiento reactivo impulsado por eventos (procesar un archivo recién subido a S3, procesar mensajes de una cola SQS, webhooks externos de facturación o geoposición) y microservicios con tráfico variable o intermitente.

---

5. ¿Cómo manejarías credenciales y secretos de conexión en AWS para una aplicación .NET?

Rta//. Mediante el principio de **Cero Texto Plano** y delegación de identidades con IAM:

1. **Almacenamiento Seguro:** Guardar cadenas de conexión, API keys y llaves privadas en **AWS Secrets Manager** con cifrado en reposo mediante **AWS KMS**.
2. **Autenticación sin Claves (IAM Roles):** La aplicación .NET Core en ECS Fargate no contiene llaves `AWS_ACCESS_KEY_ID`. Utiliza una política asignada a la tarea (**ECS Task Role**) con permisos mínimos (`secretsmanager:GetSecretValue`) sobre el ARN del secreto específico.
3. **Integración Nativa en .NET:** En el arranque de la aplicación (`Program.cs`), utilizar el proveedor de configuración de AWS (`Amazon.Extensions.Configuration.SystemsManager` o el cliente SDK de Secrets Manager) para inyectar el secreto directamente en el árbol `IConfiguration` en memoria.
4. **Inyección en Tiempo de Despliegue (Recomendada en ECS):** Configurar la directiva `secrets` en el `TaskDefinition.json` de ECS referenciando el ARN de Secrets Manager, permitiendo que el agente de ECS inyecte la variable de entorno en memoria al contenedor antes de inicializar la aplicación.

---

6. ¿Qué ventajas tiene implementar CI/CD en un equipo de desarrollo?

Rta//.

1. **Detección Temprana de Errores (Shift-Left):** La ejecución automática de linters, escaneos de seguridad estática (SAST) y pruebas unitarias/integración en cada pull request previene la introducción de bugs en ramas principales.
2. **Reducción Drástica del Time-to-Market:** Permite entregas frecuentes, pequeñas e incrementales a producción (horas en lugar de semanas).
3. **Eliminación del Error Humano:** Los pasos manuales propensos a fallos (compilación local, copiado manual de archivos, scripts ejecutados por terminal) se reemplazan por pipelines deterministas y auditables.

4. **Despliegues Libres de Tiempo de Inactividad:** Facilita estrategias de despliegue avanzadas como *Blue/Green* o *Canary*, mejorando la disponibilidad del negocio.
5. **Cultura de Responsabilidad Compartida:** Aumenta la confianza del equipo al garantizar que cualquier commit en la rama principal es potencialmente desplegable.

---

7. ¿Cómo manejarías el versionamiento y los despliegues rollback en un entorno de producción con CI/CD?

Rta//.

- **Estrategia de Versionamiento:**
  - **Versionamiento Semántico (SemVer - `MAJOR.MINOR.PATCH`):** Etiquetado automático (`Git Tags`) mediante herramientas como *GitVersion* o *Conventional Commits*.
  - **Imágenes de Contenedores Inmutables:** Etiquetar cada imagen en Amazon ECR con el hash del commit (`git-sha`) y el tag SemVer (`v1.2.3`). Queda estrictamente prohibido desplegar la etiqueta mutable `latest` en producción.
- **Estrategia de Despliegue y Rollback:**
  - **Despliegue Blue/Green o Canary (con AWS ECS + CodeDeploy / ArgoCD):** Se levanta el nuevo conjunto de contenedores (Green) y el balanceador (ALB) redirige progresivamente el tráfico (ej. 10% ➔ 50% ➔ 100%).
  - **Rollback Automático por Alarmas:** Si las métricas de CloudWatch detectan un incremento en códigos de error HTTP 5xx o latencia anómala durante el despliegue, el pipeline cancela el paso de tráfico y revierte automáticamente al ambiente anterior (Blue) en segundos sin intervención humana.
  - **Rollback de Base de Datos:** Las migraciones de base de datos deben ser **compatibles hacia atrás** (*Expand/Contract Pattern*). Nunca se eliminan o renombran columnas en la misma versión que el código nuevo; se añaden columnas opcionales primero, se despliega la API, y en una fase posterior se remueven los artefactos obsoletos.

---

8. Explica una diferencia clave entre ASP.NET MVC y ASP.NET Core MVC

Rta//.

- **Diferencia Fundamental: Arquitectura de Runtime y Multiplataforma.**
  - **ASP.NET MVC (Legacy):** Estaba fuertemente acoplado al runtime de **.NET Framework completo** y dependía estructuralmente de la biblioteca `System.Web.dll`, lo que obligaba a ejecutar exclusivamente sobre servidores **Windows e Internet Information Services (IIS)**.
  - **ASP.NET Core MVC:** Fue rediseñado desde cero como un framework modular y liviano que corre sobre el runtime multiplataforma moderno (.NET Core / .NET 8/9). Es completamente agnóstico al sistema operativo (ejecuta nativamente en **Linux, macOS y contenedores Docker ligeros** como Alpine/Chiseled), utiliza el servidor web ultrarrápido **Kestrel** y posee un pipeline de middleware basado en `RequestDelegate` con inyección de dependencias (DI) nativa de primer nivel.

---

9. ¿Cómo garantizarías la seguridad en una API REST desarrollada en .NET Core?

Rta//. A través de una estrategia de **Defensa en Profundidad (Defense in Depth)**:

1. **Autenticación y Autorización:** Implementar OAuth 2.0 / OpenID Connect con tokens Bearer JWT validados en cada petición (firma, issuer, audience y expiración), respaldados por autorización basada en roles (RBAC) o directivas por Claims/Policies.
2. **Cifrado de Comunicaciones:** Forzar HTTPS mediante HSTS (`UseHsts`) y TLS 1.3, rechazando tráfico plano HTTP.
3. **Validación Exhaustiva de Entradas:** Validar todos los modelos de entrada mediante librerías como *FluentValidation* antes de procesar la lógica de negocio, sanitizando entradas para prevenir inyecciones SQL y ataques XSS.

4. **Protección de Cabeceras y CORS:** Configuración estricta de CORS restringiendo orígenes, métodos y headers permitidos, complementado con cabeceras de seguridad (`X-Content-Type-Options: nosniff` , `X-Frame-Options: DENY` , `Content-Security-Policy`).
5. **Mitigación de Denegación de Servicio:** Implementar *Rate Limiting* nativo de .NET 8/9 (`app.UseRateLimiter`) a nivel de IP y token para evitar abusos y scraping.
6. **Gestión de Errores Segura:** Middleware global que emita respuestas estandarizadas RFC 7807 (`ProblemDetails`), evitando fugas de información sensible como Stack Traces o cadenas de conexión en entornos no-desarrollo.

---

10. ¿Cuándo preferirías usar ADO.NET sobre Entity Framework y por qué?

Rta//. Preferiría usar **ADO.NET (o micro-ORMs basados en ADO.NET como Dapper)** sobre Entity Framework en los siguientes escenarios:

1. **Operaciones Batch / Carga Masiva de Datos:** En escenarios logísticos donde se deben insertar o actualizar miles de registros de guías, tracking o paquetes en segundos (usando `SqlBulkCopy`), donde el Change Tracker y la generación de sentencias individuales de EF Core introducen una degradación crítica de CPU y memoria.
2. **Procesos Críticos de Ultra-Baja Latencia:** En transacciones de alta concurrencia donde cada microsegundo cuenta y se requiere ejecutar T-SQL altamente optimizado con lectura directa vía `SqlDataReader` sin sobrecarga de mapeo de entidades.
3. **Reportes Complejos y Consultas Analíticas Avanzadas:** Consultas que involucran CTEs jerárquicos, hints específicos (`WITH (NOLOCK, INDEX(...))`), llamadas a Procedimientos Almacenados complejos o tablas temporales dinámicas donde EF Core genera consultas subóptimas o no ofrece soporte idiomático directo.

PARTE 2: Solución al Caso de Estudio (Compañía de Logística ZYX)

1. Diagnóstico de la Problemática Actual

Problema Identificado en ZYX	Causa Raíz Técnica	Impacto en la Operación y Rentabilidad
Recursos Insuficientes On-Premise	Servidores físicos con capacidad finita no elástica ante picos de admisión logística.	Caída del sistema en temporadas pico (Black Friday, festivos), pérdida de transacciones y cuellos de botella en admisión.
Degradación de Base de Datos	La misma base de datos procesa transacciones de admisión mientras genera reportes gerenciales pesados.	Bloqueos de tablas (*Lock Escalation*), saturación de I/O de disco y caídas generales del Core transaccional.
Integración Deficiente con Ventas y Logística	Acoplamiento síncrono punto a punto o transferencias por archivos batch sin validación contractual.	Inconsistencia de inventario, retrasos de actualización de envíos y pérdida de visibilidad de trazabilidad.
Retrasos en Proveedores Externos (Geo/Facturación)	Llamadas HTTP síncronas bloqueantes a APIs externas durante el flujo de admisión.	La lentitud o caída del proveedor de geoposición/facturación congela el proceso de admisión del usuario final (*Cascade Failure*).
Errores Humanos en Admisión	Interfaz monolítica obsoleta, sin validaciones reactivas en vivo ni diseño ergonómico guiado por pasos (*Wizard*).	Retrabajo operativo, paquetes con direcciones erróneas, retrasos de entrega y costos adicionales por devolución.

2. Propuesta de Arquitectura de Modernización (Nube Híbrida AWS + OnPremise)

La solución adopta el patrón **CQRS (Command Query Responsibility Segregation)**, una arquitectura **Orientada a Eventos (Event-Driven)** y la modernización de la experiencia de usuario.

2.1 Diagrama de Arquitectura de la Nueva Plataforma (Mermaid)

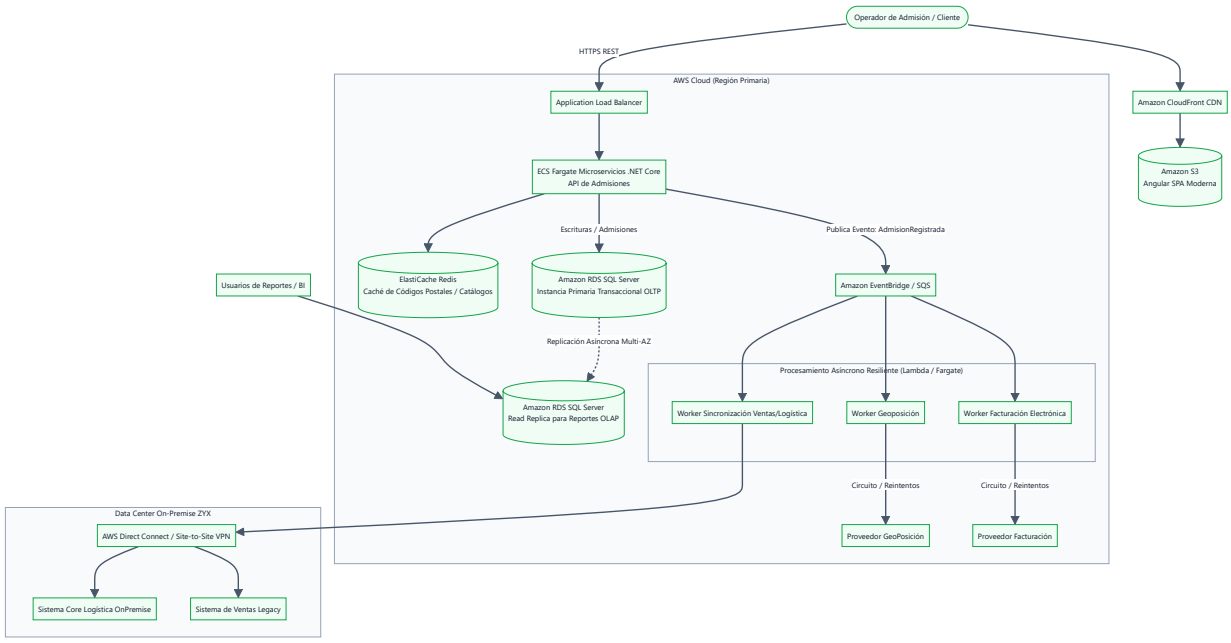


DIAGRAMA DE ARQUITECTURA / MODELO TÉCNICO

3. Plan de Mitigación Detallado

3.1 Solución a Recursos Insuficientes y Escalabilidad

- **Migración a AWS ECS Fargate (.NET Core):** Sustituir la infraestructura rígida On-Premise por contenedores serverless en subredes privadas. Se configuran políticas de escalado automático horizontal (*Target Tracking*) para aumentar dinámicamente tareas ante incrementos repentinos de admisiones.

3.2 Solución a la Degradación de la Base de Datos (CQRS de Persistencia)

- **Separación Transaccional vs Lectura/Reportes:**
  - **Comandos (Escrituras de Admisión):** Dirigidas exclusivamente al nodo primario de **Amazon RDS for SQL Server**.
  - **Consultas y Reportes:** Dirigidas a una **Read Replica** dedicada. Ningún reporte gerencial o analítico tiene permisos de ejecución sobre el nodo primario transaccional, eliminando el 100% de los cuellos de botella por contención de bloqueos.
  - **Caché en Memoria:** Implementación de **ElastiCache Redis** para catálogos estáticos (tarifas, ciudades, departamentos, terminales), reduciendo lecturas a disco.

3.3 Solución a los Retrasos de Proveedores Externos (Patrón Outbox + Colas)

- **Desacoplamiento Asíncrono:** La admisión del envío **no espera** la respuesta de los proveedores externos.
  1. El operador registra la admisión y recibe confirmación inmediata en menos de 300 ms.
  2. La API emite un evento `AdmisionCreada` hacia una cola **Amazon SQS**.

3. Workers independientes procesan la geocodificación y la facturación electrónica en segundo plano.
4. **Patrón Circuit Breaker y Reintentos Exponenciales (Polly):** Si el proveedor de geocodificación se degrada o cae, las peticiones se encolan con reintentos y *Dead Letter Queues* (DLQ), sin congelar la operación del operario en ventanilla.

3.4 Solución a la Integración Deficiente con Ventas y Logística

- **Broker Central de Eventos (EventBridge / Kafka):** Se establece un contrato canónico de eventos de dominio en JSON. Cuando ocurre un cambio en admisión, ventas o logística, los sistemas se sincronizan mediante suscripciones a eventos a través de **AWS Direct Connect / VPN Site-to-Site**, garantizando consistencia eventual y eliminando discrepancias manuales.

3.5 Solución a los Errores Humanos (Modernización de Frontend)

- **Single Page Application en Angular con Signals:**
  - **Diseño Orientado a Pasos (Wizard Ergonómico):** División del proceso en: 1. Remitente/Destinatario ➔ 2. Paquete y Tarifación ➔ 3. Confirmación.
  - **Validación Reactiva en Vivo:** Validación en tiempo real de campos (código postal, pesos volumétricos, teléfonos) evitando envíos incompletos.
  - **Autocompletado Inteligente y Geocodificación en Cliente:** Normalización de direcciones asistida con debounce para reducir errores tipográficos en un 90%.

---

4. Hoja de Ruta de Implementación (Roadmap)

```
Fase 1: Quick Wins (Semanas 1-4)
├─ Implementación de Read Replica en BD para aislar reportes pesados.
└─ Optimización de índices críticos y activación de RCSI en SQL Server.

Fase 2: Prototipado y Piloto Cloud (Semanas 5-8)
├─ Construcción de la SPA Angular (Wizard ergonómico de admisión).
├─ Despliegue de la API REST .NET Core en ECS Fargate con Terraform.
└─ Desacoplamiento asíncrono con colas SQS para facturación y geoposición.

Fase 3: Conectividad Híbrida y Despliegue Total (Semanas 9-12)
├─ Interconexión de eventos hacia sistemas de ventas y logística OnPremise.
├─ Monitoreo integral con CloudWatch y trazabilidad distribuida con X-Ray.
└─ Salida a producción gradual mediante despliegue Canary por sedes piloto.
```

03

Arquitectura del Backend: Hexagonal, Mediador Nativo & CQRS (.NET 8)

ARQUITECTURA DE SOFTWARE BACKEND

Arquitectura del Backend: Especificación Técnica y Justificación de Diseño

Sistema: Plataforma de Registro Académico y Selección de Materias
Plataforma: .NET 8 (C# 12)
Organización: Inter Rapidísimo S.A.
Estado: Producción / Validado con 100% de Pruebas Unitarias

1. Visión General de la Arquitectura

El backend implementa una **Arquitectura Hexagonal (Ports & Adapters)** fusionada con los principios de **Clean Architecture** y una distribución canónica del patrón **CQRS (Command Query Responsibility Segregation)**.

El diseño prioriza el desacoplamiento estricto, la independencia de librerías externas para componentes neurálgicos (como el mediador y las validaciones), la alta testabilidad y el control de flujo funcional mediante el **Result Pattern** (*Railway Oriented Programming*).

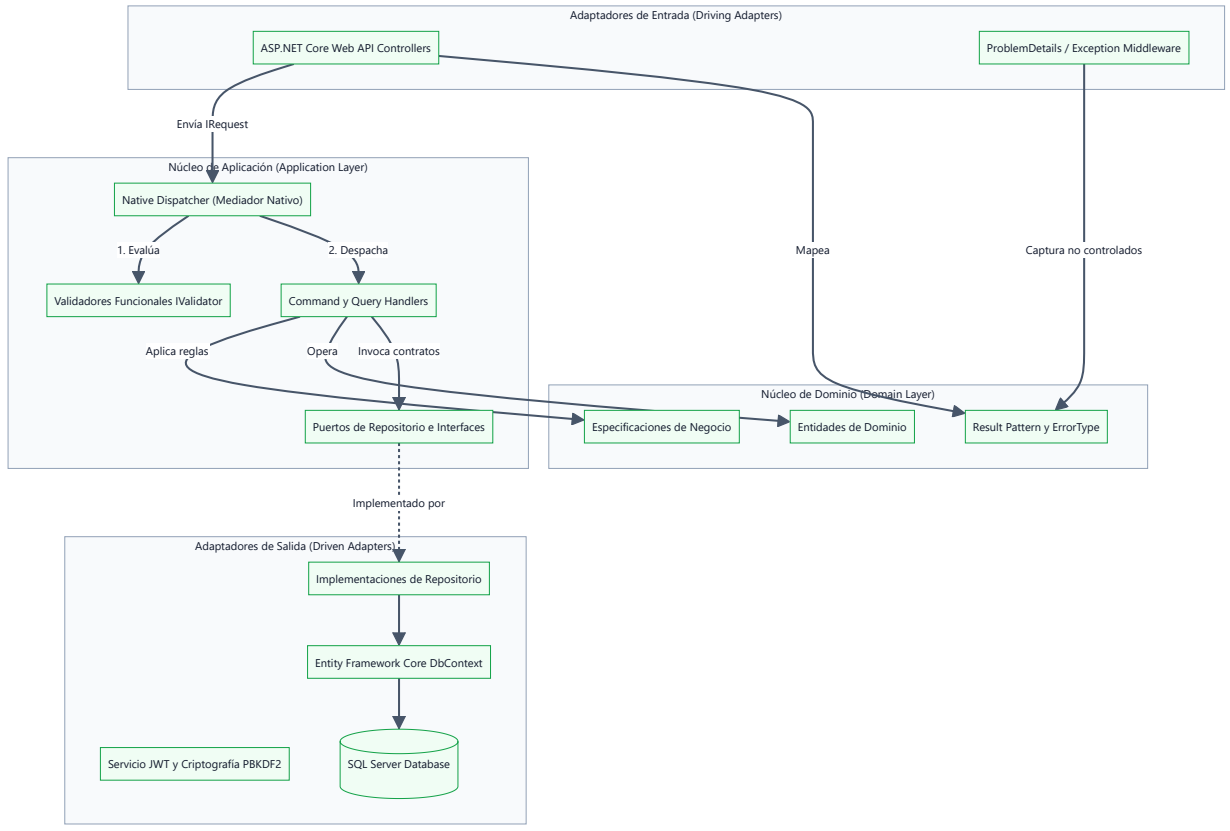


DIAGRAMA DE ARQUITECTURA / MODELO TÉCNICO

Principios Fundamentales

- **Inversión de Dependencias (DIP):** Las capas internas (`Domain` y `Application`) nunca dependen de las capas externas (`Infrastructure` o `Api`). Las capas externas implementan los puertos definidos en el núcleo.
- **Separación de Responsabilidades (SoC):** Cada clase y capa posee un único propósito delimitado. Los controladores HTTP solo gestionan transporte y códigos de estado; los handlers coordinan casos de uso; las entidades encapsulan estado e invariantes.
- **Zero External Dependencies en el Núcleo:** El mediador CQRS, los validadores y las abstracciones de negocio se implementaron de forma nativa sin depender de paquetes NuGet como MediatR o FluentValidation.

---

2. Topología de la Solución y Distribución de Carpetas

La solución sigue una convención física y lógica estricta que refleja el flujo de dependencias:

```
src/backend/
├── Core/
│   ├── Domain/                    # Capa más interna: Reglas puras, Entidades y Primitivas
│   │   ├── Common/                # Clases base, especificaciones y Result<T>
│   │   ├── Entities/              # Modelos del dominio con estado e identidad
│   │   └── Specifications/        # Reglas complejas encapsuladas (Specification Pattern)
│   │
│   └── Application/               # Casos de uso de negocio (CQRS Canónico)
│       ├── Common/                # Interfaces del mediador y validadores nativos
│       │   ├── Dispatcher/        # IRequest, IRequestHandler, IDispatcher, NativeDispatcher
│       │   └── Validation/        # IValidator<TRequest>
│       ├── DTOs/                  # Contratos de transferencia de datos de entrada/salida
│       ├── Extension/             # Inyección de dependencias modular (DependencyInjection.cs)
│       ├── Features/              # Casos de uso organizados por Slices Funcionales
│       │   ├── Admin/             # Comandos administrativos (Create, Delete)
│       │   ├── Auth/              # Comandos de autenticación (Login, Register)
│       │   ├── Catalog/           # Consultas de materias y perfiles (GetCourses, etc.)
│       │   └── Enrollments/       # Inscripción y cancelación de materias (Enroll, Drop)
│       └── Ports/                 # Puertos secundarios: Contratos de repositorios y servicios
│
├── Adapters/
│   ├── Infrastructure/            # Adaptador secundario: Base de datos y servicios externos
│   │   ├── Authentication/        # Generación de tokens JWT y hashing PBKDF2
│   │   ├── Extension/             # Inyección de dependencias modular (DependencyInjection.cs)
│   │   ├── Persistence/           # EF Core DbContext, Semillas e Inicializador
│   │   │   ├── Configurations/    # Mapeos Fluent API independientes por entidad
│   │   │   └── Repositories/      # Implementación concreta de los puertos del núcleo
│   │   │
│   └── Api/                       # Adaptador primario: Entrada HTTP / REST
│       ├── Common/                # Extensiones de traducción (ErrorTypeExtensions)
│       ├── Controllers/           # Controladores delgados derivados de BaseApiController
│       ├── Extension/             # Inyección de dependencias de API (DependencyInjection.cs)
│       ├── Middleware/            # Manejo global de excepciones inesperadas
│       └── Program.cs             # Punto de entrada ultraligero y orquestador del pipeline
│
└── tests/
    ├── Domain.UnitTests/          # Pruebas unitarias de especificaciones y Result<T>
    └── Application.UnitTests/     # Pruebas unitarias de Handlers y flujos de negocio
```

---

3. Desglose Técnico por Capas y Justificaciones

3.1. Capa de Dominio (`Core/Domain`)

Es el núcleo del sistema, libre de cualquier dependencia externa, framework de infraestructura o referencias a bases de datos.

Componentes Principales:

1. **Entidades (`Domain/Entities/`):**
   - Representan los conceptos clave: `Student`, `Professor`, `Course`, `Enrollment`, `User`.
   - Heredan de `BaseEntity`, la cual garantiza columnas canónicas de auditoría: `Id`, `CreatedAtUtc`, `UpdatedAtUtc` e `IsDeleted` (Soft Delete).
2. **Patrón Specification (`Domain/Specifications/`):**
   - Las reglas de negocio complejas no se dispersan en servicios o controladores. Se encapsulan mediante la interfaz `ISpecification<T>`:
     - `MaxCoursesSpecification`: Garantiza que ningún estudiante exceda el límite de 3 materias inscritas (9 créditos).
     - `UniqueProfessorSpecification`: Aplica la regla innegociable que prohíbe inscribir dos materias dictadas por el mismo profesor.
     - `NotAlreadyEnrolledSpecification`: Impide la inscripción duplicada de la misma materia activa.
   - **Justificación:** Centraliza la lógica de negocio, facilita su reutilización y permite probar las reglas de forma aislada sin levantar contextos de base de datos ni mocks complejos.
3. **Primitiva Funcional `Result<T>` y `ErrorType` (`Domain/Common/Result.cs`):**
   - Sustituye el lanzamiento de excepciones para control de flujo por el patrón de resultados.

3.2. Capa de Aplicación (`Core/Application`)

Implementa la orquestación de los casos de uso del sistema utilizando una distribución canónica de **CQRS Vertical Slice**.

3.2.1. Organización de Features (Cohesión Vertical)

En lugar de agrupar todos los comandos en una carpeta masiva y todos los manejadores en otra, cada funcionalidad reside en su propio subdirectorio:

```
Features/Enrollments/Commands/Enroll/
├── EnrollCourseCommand.cs           # Definición inmutable de la solicitud (IRequest<Result<T>>)
├── EnrollCourseCommandValidator.cs  # Reglas de validación previas a la ejecución
└── EnrollCourseCommandHandler.cs    # Lógica de orquestación del caso de uso
```

> [!NOTE] **Justificación Arquitectónica:** Este enfoque maximiza el principio de **alta cohesión**. Si se requiere modificar o auditar el caso de uso de inscripción, todos los artefactos relacionados (contrato, validación y ejecución) están agrupados en el mismo contexto físico y conceptual, minimizando la dispersión de código y el riesgo de regresiones.

3.2.2. Mediador Nativo (`NativeDispatcher`)

Se diseñó un despachador interno basado en el patrón Mediador:

- `IDispatcher`: Expone el contrato `Task<TResponse> SendAsync<TResponse>(IRequest<TResponse> request)`.
- `NativeDispatcher`: Resuelve en tiempo de ejecución el manejador correspondiente desde el contenedor de dependencias de .NET.
- **Pipeline Behavior de Validación:** Antes de invocar a cualquier manejador, el despachador consulta si existe un `IValidator<TRequest>` registrado para la solicitud. Si el validador retorna un fallo (`!result.IsSuccess`), el flujo se **cortocircuita inmediatamente**, retornando `Result<TResponse>.Failure(...)` sin tocar el Handler ni la base de datos.

- **Justificación:** Elimina la dependencia de paquetes de terceros (MediatR), reduce la huella de memoria del runtime y otorga control total sobre la cadena de intercepción.

3.2.3. Puertos Secundarios (`Application/Ports/`)

Define interfaces estrictas para la persistencia y servicios transversales:

- `IStudentRepository` , `ICourseRepository` , `IProfessorRepository` , `IEnrollmentRepository` , `IUserRepository` .
- `IUnitOfWork` : Garantiza consistencia transaccional mediante `CommitAsync()` .
- `IPasswordHasher` , `ITokenService` .
- **Justificación:** Desacopla la lógica de negocio del motor de persistencia subyacente. Los casos de uso operan contra abstracciones puras, permitiendo cambiar el proveedor de base de datos o implementar simuladores de prueba sin alterar una sola línea de lógica.

3.3. Capa de Infraestructura (`Adapters/Infrastructure`)

Provee las implementaciones concretas para los puertos requeridos por la capa de aplicación.

Componentes Principales:

1. **Configuraciones Fluent API Independientes (`Persistence/Configurations/`):**
   - Cada entidad tiene su propio archivo de configuración (`CourseConfiguration.cs` , `EnrollmentConfiguration.cs` , etc.).
   - Establece restricciones de unicidad, longitudes máximas, relaciones foráneas e índices filtrados para exclusiones lógicas (`IsDeleted = 0`).
2. **Repositorios Individuales (`Persistence/Repositories/`):**
   - Cada puerto del núcleo cuenta con una clase de implementación dedicada (ej. `CourseRepository.cs`), evitando repositorios genéricos anémicos que limitan la optimización de consultas SQL.
3. **Criptografía y Autenticación:**
   - `Pbkdf2PasswordHasher` : Hashing criptográfico seguro utilizando sal aleatoria e iteraciones SHA-256.
   - `JwtTokenService` : Emisión de tokens firmados bajo el estándar RFC 7519 con roles de usuario (`Estudiante` , `Administrador`).
4. **Semillas y Creación (`DbInitializer.cs`):**
   - Carga controlada de datos iniciales requeridos: 5 profesores, 10 materias académicas y credenciales administrativas por defecto.

3.4. Capa de API (`Adapters/Api`)

Es el adaptador primario que expone los casos de uso a través del protocolo HTTP / REST.

3.4.1. Controlador Base y Mapeo Dinámico de Códigos HTTP

Los controladores heredan de `BaseApiController` , el cual centraliza el método de respuesta `ToActionResult` :

```
protected IActionResult ToActionResult<T>(Result<T> result, int successStatusCode = StatusCodes.Status200OK)
{
    var statusCode = result.IsSuccess ? successStatusCode : result.ErrorType.ToHttpStatusCode();
```

```
    return StatusCode(statusCode, result);
}
```

El método de extensión `ErrorTypeExtensions` traduce la semántica del dominio a códigos estándar de la IETF:

`ErrorType` (Dominio)	Código HTTP	Significado REST
`None`	`200 OK` / `201 Created`	Operación ejecutada exitosamente
`Validation` / `BadRequest`	`400 Bad Request`	Violación de regla de negocio o formato de entrada
`Unauthorized`	`401 Unauthorized`	Credenciales inválidas o ausencia de token de autenticación
`Forbidden`	`403 Forbidden`	Permisos insuficientes para el recurso solicitado
`NotFound`	`404 Not Found`	Recurso o entidad inexistente en el sistema
`Conflict`	`409 Conflict`	Conflicto de estado (duplicidad de materias, profesor repetido)
`InternalServerError`	`500 Internal Server Error`	Excepción técnica no controlada del sistema

3.4.2. Middleware Global de Excepciones Inesperadas

`ProblemDetailsExceptionMiddleware` actúa como red de seguridad final. No captura errores de negocio (ya controlados por `Result<T>`), sino fallas imprevistas de infraestructura (caída del servidor SQL, timeout de red). En estos casos, serializa un `Result<object?>.Failure(...)` con código 500, impidiendo que trazas internas de la aplicación se filtren al cliente.

---

4. Justificación del Patrón Result frente al Uso de Excepciones

La sustitución total de excepciones de dominio por la clase `Result<T>` responde a decisiones de ingeniería de software de alto rendimiento:

```
Flujo Tradicional (Antipatrón Exception-Driven Flow):
Request ──> Handler ──> throw DomainException ──> Unwinding de Stack Trace ──> Middleware ──> Response HTTP

Flujo Funcional Implementado (Result Pattern):
Request ──> Handler ──> return Result<T>.Failure(...) ──> ToActionResult ──> Response HTTP (Instantáneo)
```

Argumentos Técnicos:

1. **Cero Penalización de Rendimiento:** El lanzamiento y captura de excepciones en el CLR de .NET requiere pausar la ejecución del hilo para inspeccionar la pila de llamadas (*stack unwinding*). En escenarios de alta concurrencia o errores frecuentes de validación, esto genera una sobrecarga computacional severa. `Result<T>` es un retorno de objeto regular en memoria sin coste adicional.
2. **Firma de Métodos Explícita y Predictible:** Una firma `Task<Result<AuthResponse>> HandleAsync(...)` documenta formalmente en el sistema de tipos de C# que la operación puede fallar y obliga al consumidor a evaluar el éxito o fracaso, eliminando efectos secundarios ocultos.
3. **Estandarización del Contrato JSON:** El 100% de las respuestas de la API devuelven una estructura uniforme:

```
{
  "isSuccess": false,
  "message": "Ya existe una materia registrada con ese código.",
  "errors": [],
  "result": null,
  "errorType": 6
}
```

---

5. Modularidad de la Inyección de Dependencias

Para evitar la degradación de `Program.cs` en un archivo monolítico (*God File*), cada capa expone su propia clase estática `DependencyInjection.cs`:

```
// Program.cs queda reducido a 30 líneas de alta legibilidad:
builder.Services.AddApplicationDispatcher();               // Capa Application
builder.Services.AddInfrastructureServices(builder.Configuration); // Capa Infrastructure
builder.Services.AddApiServices(builder.Configuration);           // Capa Api
```

Registro Determinista sin Reflexión Insegura

En `Application/Extension/DependencyInjection.cs`, el registro de Handlers y Validadores se realiza de forma **explícita y determinista**:

- **Problema resuelto:** El uso de escaneo por reflexión dinámico (`Assembly.GetTypes()`) suele fallar en entornos corporativos o Windows 11 con directivas de integridad de código (*Smart App Control*) debido al bloqueo preventivo de DLLs locales no firmadas (código de error `0x800711C7`).
- **Solución:** El registro directo previene al 100% las excepciones de tipo `ReflectionTypeLoadException` , acelera el tiempo de arranque en frío (*cold start*) y garantiza un comportamiento idéntico en desarrollo y producción.

---

6. Estrategia de Pruebas Automatizadas

La solución cuenta con una suite integral de pruebas unitarias implementadas con **xUnit**, **Moq** y **FluentAssertions**:

- **`Domain.UnitTests`** :
  - Evaluación exhaustiva de las especificaciones (`MaxCoursesSpecification` , `UniqueProfessorSpecification` , `NotAlreadyEnrolledSpecification`).
  - Comprobación del comportamiento de las fábricas estáticas `Result<T>.Success` y `Result<T>.Failure` .
- **`Application.UnitTests`** :
  - Cobertura de flujos completos en `RegisterStudentCommandHandler` , `LoginCommandHandler` , `EnrollCourseCommandHandler` , `DropCourseCommandHandler` , etc.
  - Verificación de que las respuestas de fallo retornan el `ErrorType` esperado sin arrojar excepciones.
  - Validación de la regla de privacidad por diseño en las consultas de compañeros y estudiantes públicos.

---

7. Resumen de Decisiones de Arquitectura

Decisión	Enfoque Implementado	Beneficio Directo
Estilo Arquitectónico	Hexagonal (Ports & Adapters)	Independencia de base de datos, frameworks y alta testabilidad
Separación de Casos de Uso	CQRS Canónico en Vertical Slices	Alta cohesión de comandos, consultas, validadores y manejadores
Manejo de Errores	Result Pattern con `ErrorType`	Cero costo de excepciones de pila y contratos JSON uniformes
Mediador CQRS	Despachador Nativo (`NativeDispatcher`)	Sin dependencias externas de NuGet; control total del ciclo de vida
Validaciones	`IValidator<T>` con cortocircuito en pipeline	Evaluación previa sin alcanzar los repositorios ni saturar la BD
Reglas de Negocio	Specification Pattern (`ISpecification<T>`)	Lógica compleja aislada, testeable y reutilizable
Transaccionalidad	Unit of Work explícito	Atomicidad en operaciones que involucran múltiples agregados
Inyección de Dependencias	Extensiones modulares por capa	`Program.cs` limpio, arranque determinista e inmune a fallos de reflexión

04

Arquitectura del Frontend: Zoneless, Signals & Standalone (Angular 21)

ARQUITECTURA DE SOFTWARE FRONTEND

Arquitectura del Frontend: Especificación Técnica y Justificación de Diseño

Sistema: Plataforma de Registro Académico y Selección de Materias
Tecnología: Angular 21 (Node 24 / TypeScript 5.9)
Organización: Inter Rapidísimo S.A.
Estado: Producción / Compilación Validada (0 advertencias, 0 errores)

---

1. Visión General de la Arquitectura

El frontend ha sido diseñado bajo los estándares modernos de **Angular 21**, adoptando una **Arquitectura Reactiva Zoneless** basada en **Angular Signals** y **Standalone Components**.

El diseño prioriza el desacoplamiento estricto, la eliminación de paquetes redundantes (como `zone.js`), la segregación de roles mediante Guardas Funcionales, el consumo transparente de la envoltura funcional `Result<T>` del backend y una distribución modular de componentes por dominio (*Feature-Driven Architecture*).

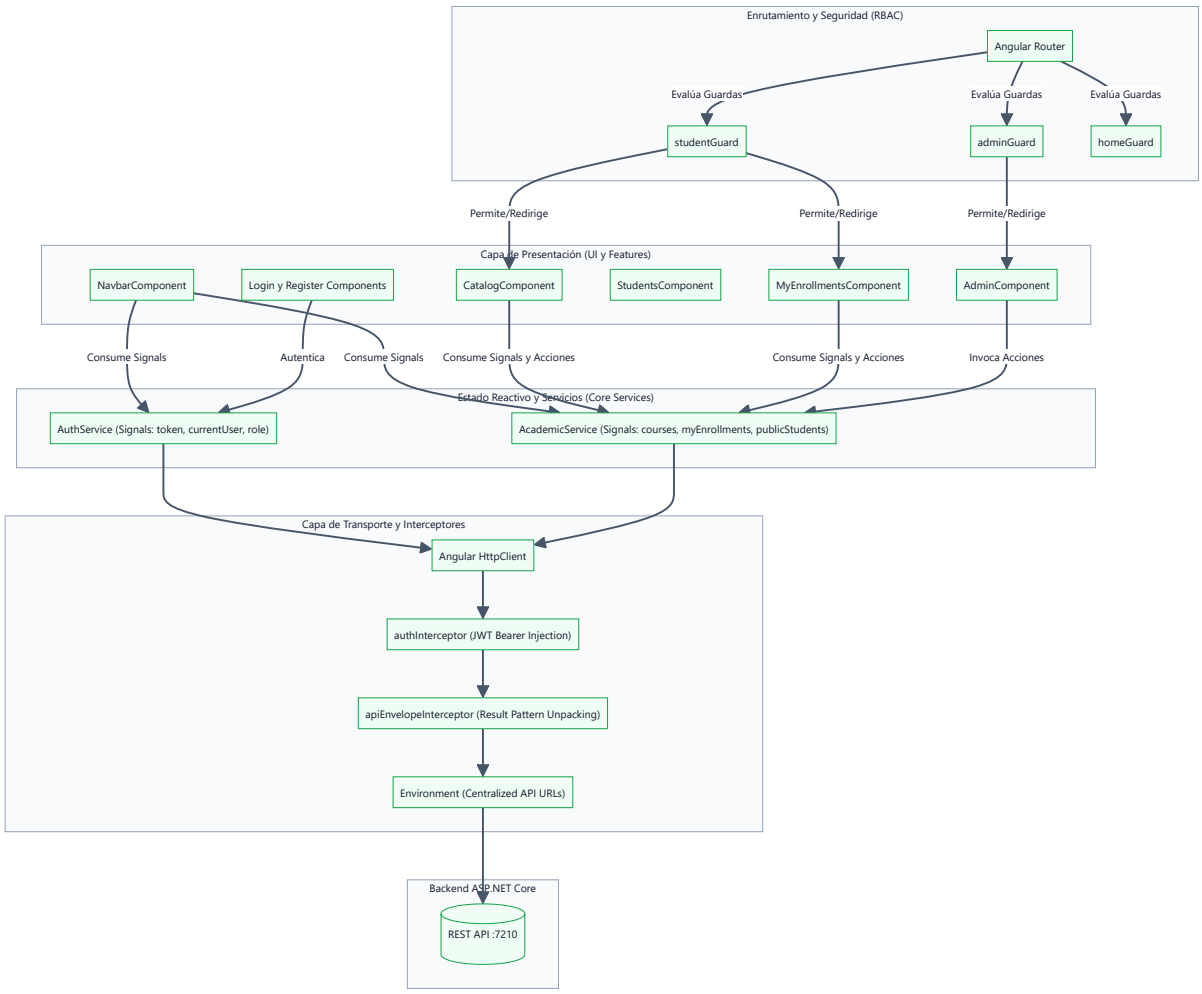


DIAGRAMA DE ARQUITECTURA / MODELO TÉCNICO

Principios Fundamentales

- **Arquitectura Zoneless Nativa:** Se implementa `provideZonelessChangeDetection()`, eliminando por completo la necesidad de `zone.js`. Las actualizaciones del DOM ocurren de manera atómica y eficiente mediante Signals, sin sobrecoste de *monkey-patching* global en microtareas del navegador.
- **Componentes Standalone Puros:** Eliminación definitiva de los antiguos `NgModule`. Cada componente declara explícitamente sus dependencias en `imports`, facilitando el *tree-shaking* y acelerando los tiempos de compilación.
- **Separación Estricta de Interfaz (HTML/CSS/TS):** Cada componente cuenta con sus tres archivos independientes según las directrices oficiales de Angular, garantizando mantenibilidad y claridad.
- **Segregación de Roles (RBAC):** La interfaz y el enrutador discriminan estrictamente entre el rol `Estudiante` y `Administrador`. Un administrador no tiene acceso ni visualización del catálogo ni de la inscripción de asignaturas.

---

2. Topología de la Solución y Distribución de Carpetas

La estructura física del proyecto refleja una organización **Modular por Capas y Funcionalidades**:

```
src/frontend/src/
├── app/
│   ├── core/                              # Núcleo transversal de la aplicación (Singletons y Contratos)
│   │   ├── guards/                        # Guardas funcionales de protección de rutas (RBAC)
│   │   │   └── auth.guard.ts              # studentGuard, adminGuard, homeGuard, authGuard
│   │   │
│   │   ├── interceptors/                  # Pipeline HTTP de salida y entrada
│   │   │   ├── auth.interceptor.ts        # Inyección del token Bearer JWT
│   │   │   └── api-envelope.interceptor.ts  # Desempaquetado del Result<T> del backend
│   │   │
│   │   ├── models/                        # Definición estricta de interfaces y contratos
│   │   │   ├── auth.model.ts              # UserProfileDto, AuthResponse
│   │   │   ├── classmate.model.ts         # ClassmateDto (Políticas de privacidad)
│   │   │   ├── course.model.ts            # CourseDto
│   │   │   ├── enrollment.model.ts        # EnrolledCourseItemDto, StudentEnrollmentSummaryDto
│   │   │   ├── problem-details.model.ts   # ProblemDetails (RFC 7807)
│   │   │   ├── professor.model.ts         # ProfessorDto
│   │   │   ├── result.model.ts            # Result<T> y ErrorType (Railway Oriented Programming)
│   │   │   ├── student.model.ts           # PublicStudentDto, StudentProfileDto
│   │   │   ├── index.ts                   # Barrel export unificado
│   │   │   └── models.ts                  # Reexportación para compatibilidad
│   │   │
│   │   └── services/                      # Servicios de negocio con reactividad basada en Signals
│   │       ├── auth.service.ts            # Sesión, Login, Register, Logout y Signals de usuario
│   │       └── academic.service.ts        # Gestión de catálogo, matrículas y administración
│   │
│   ├── features/                          # Módulos funcionales de la aplicación
│   │   ├── admin/                         # Panel administrativo de docentes y asignaturas
│   │   │   ├── admin.component.ts
│   │   │   ├── admin.component.html
│   │   │   └── admin.component.css
│   │   │
│   │   ├── auth/                          # Autenticación y registro desacoplado por vistas
│   │   │   ├── login/
│   │   │   │   ├── login.component.ts
│   │   │   │   ├── login.component.html
│   │   │   │   └── login.component.css
│   │   │   └── register/
│   │   │       ├── register.component.ts
│   │   │       ├── register.component.html
│   │   │       └── register.component.css
│   │   │
│   │   ├── catalog/                       # Catálogo de 10 materias con validación en tiempo real
│   │   │   ├── catalog.component.ts
│   │   │   ├── catalog.component.html
│   │   │   └── catalog.component.css
│   │   │
```

```
│   │   ├── enrollments/                    # Consulta de inscripciones y compañeros de clase
│   │   │   ├── my-enrollments.component.ts
│   │   │   ├── my-enrollments.component.html
│   │   │   └── my-enrollments.component.css
│   │   │
│   │   └── students/                       # Directorio público institucional de alumnos
│   │       ├── students.component.ts
│   │       ├── students.component.html
│   │       └── students.component.css
│   │
│   ├── shared/                             # Componentes visuales transversales
│   │   └── navbar/
│   │       ├── navbar.component.ts
│   │       ├── navbar.component.html
│   │       └── navbar.component.css
│   │
│   ├── app.config.ts                       # Configuración global de la aplicación (Zoneless, Router, HTTP)
│   ├── app.routes.ts                       # Tabla de enrutamiento y asignación de Guardas
│   ├── app.ts                              # Componente raíz (Shell)
│   ├── app.html
│   └── app.css
│
├── environments/                           # Variables de entorno centralizadas
│   ├── environment.ts                      # Configuración de Producción
│   └── environment.development.ts          # Configuración de Desarrollo Local (:7210)
│
├── styles.css                              # Estilos globales y clases comunes centralizadas
└── main.ts                                 # Punto de entrada de bootstrapping
```

---

3. Desglose Técnico por Capas y Justificaciones

3.1. Arquitectura Zoneless y Gestión de Estado con Signals

En Angular convencional, el framework depende de `zone.js` para detectar cuándo un evento asíncrono (como un clic o una respuesta HTTP) ha terminado, ejecutando una verificación completa y descendente en todo el árbol de componentes (*Dirty Checking*).

¿Por qué adoptamos Zoneless en esta solución?

1. **Rendimiento de Renderizado Quirúrgico:** Al usar `provideZonelessChangeDetection()`, Angular no intercepta las APIs del navegador. Los cambios son notificados exclusivamente por los **Signals** (`signal` y `computed`). Únicamente las partes del DOM que leen dicho signal son actualizadas.
2. **Cero Fugas de Memoria (*Memory Leaks*):** En lugar de gestionar complejas cadenas de `Subscription`, `takeUntilDestroyed` o `unsubscribe` manuales en cada componente, los Signals encapsulan valores sincrónicos reactivos.
3. **Cálculo Derivado en Tiempo Real:** En `CatalogComponent`, las reglas de negocio de créditos y profesores se calculan instantáneamente:

   ```
   readonly currentCount = computed(() => this.academicService.myEnrollments()?.enrolledCoursesCount || 0);
   readonly currentCredits = computed(() => this.academicService.myEnrollments()?.totalCredits || 0);
   readonly isMaxReached = computed(() => this.currentCount() >= 3);

   readonly enrolledProfessorIds = computed(() =>
     this.academicService.myEnrollments()?.courses?.map(c => c.professorId) || []
   );
   ```

   Si el estudiante inscribe una materia, el `signal` de matrículas se actualiza y automáticamente:

   - El contador pasa de `0/9` a `3/9`.
   - Todas las materias del mismo docente se inhabilitan en la interfaz instantáneamente mediante `isConflict(course)`.

3.2. Segregación de Roles y Control de Acceso (RBAC)

El sistema distingue dos perfiles de usuario con naturalezas operativas completamente distintas:

Perfil	Alcance Permitido	Vistas Prohibidas
Estudiante	Consultar catálogo, inscribir materias (hasta 3/9 créditos sin repetir docente), retirar materias, ver sus compañeros de curso y directorio institucional.	Panel Administrativo (`/admin`).
Administrador	Administrar docentes (crear, desactivar), administrar materias ofertadas (crear, desactivar) y consultar directorio de estudiantes.	Catálogo de inscripción (`/catalog`), Mis materias (`/my-enrollments`).

Implementación Técnica:

1. **Guardas Funcionales (`core/guards/auth.guard.ts`):**
   - `studentGuard` : Si el usuario autenticado tiene rol de Administrador, el guarda intercepta la navegación y redirige automáticamente a `/admin` :

     ```
     export const studentGuard: CanActivateFn = () => {
       const auth = inject(AuthService);
       const router = inject(Router);
       if (!auth.isAuthenticated()) return router.parseUrl('/login');
       if (auth.isAdmin()) return router.parseUrl('/admin');
       return true;
     };
     ```

   - `adminGuard` : Si un Estudiante intenta acceder a `/admin` , es desviado a `/catalog` .
   - `homeGuard` : En la ruta raíz (`/`), redirige de forma inteligente según el estado de sesión: Administrador a `/admin` , Estudiante a `/catalog` , o no autenticado a `/login` .
2. **Barra de Navegación Condicional (`navbar.component.html`):**
   - El menú "Catálogo" y "Mis Materias" está blindado con `@if (authService.isStudent())` . El Administrador no tiene visibilidad de dichas opciones.

3.3. Integración con el Result Pattern del Backend

El backend no expone entidades directamente ni utiliza excepciones para control de flujo, sino que encapsula el 100% de las respuestas en `Result<T>` :

```
{
  "isSuccess": true,
  "message": null,
  "errors": [],
  "result": { ... },
  "errorType": 0
}
```

Pipeline de Interceptores HTTP

1. **`authInterceptor` :** Lee el signal `authService.token()` y, si existe, clona la solicitud HTTP inyectando la cabecera `Authorization: Bearer {token}` .

2. `apiEnvelopeInterceptor`: Intercepta la respuesta exitosa `HttpResponse`. Si el cuerpo contiene la estructura `Result<T>`, **desempaqueta automáticamente la propiedad** `result`:

```
export const apiEnvelopeInterceptor: HttpInterceptorFn = (req, next) => {
  return next(req).pipe(
    map(event => {
      if (event instanceof HttpResponse) {
        const body = event.body as Result<unknown>;
        if (body && typeof body === 'object' && 'isSuccess' in body && 'result' in body) {
          return event.clone({ body: body.result });
        }
      }
      return event;
    })
  );
};
```

   - **Justificación:** Los servicios frontend (`AcademicService`, `AuthService`) operan directamente con los tipos de dominio fuertemente tipados (`CourseDto[]`, `StudentEnrollmentSummaryDto`), abstrayendo por completo la envoltura de transporte.

3.4. Modularización de Modelos y Barrel Exports

Anteriormente concentrados en un único archivo monolítico, la capa `core/models/` se modularizó en 8 archivos especializados:

- Cada modelo reside en su propio archivo semántico (`course.model.ts`, `enrollment.model.ts`, etc.).
- El archivo `index.ts` centraliza las exportaciones (*Barrel Pattern*), permitiendo importaciones limpias y estructuradas sin riesgo de dependencias circulares.

3.5. Centralización de Estilos y Diseño Visual

Para evitar duplicidad de reglas CSS en los componentes, se estableció una jerarquía clara:

1. **Estilos Globales y Compartidos (`src/styles.css`):**
   - Tipografía y encabezados (`.page-title`, `.page-subtitle`).
   - Alertas de estado (`.alert-success`, `.alert-error`, `.privacy-alert`).
   - Controles de formulario (`.form-group`, labels, inputs y focus corporativo).
   - Botones de acción estándar (`.btn-submit`, `.btn-deactivate`, `.btn-drop`).
   - Contenedores de datos (`.table-card`, `.footer-links`).
2. **Estilos Específicos de Componente:**
   - Cada archivo `.component.css` retiene únicamente el layout propio de su vista (grids de materias, modales flotantes, insignias de estado).
   - **Beneficio:** Reducción drástica del tamaño total del bundle y coherencia estética en toda la suite.

3.6. Gestión de Variables de Entorno

Siguiendo el estándar de Angular 21, la configuración se desacopla entre entornos:

- `src/environments/environment.development.ts`: Apunta a `https://localhost:7210/api/v1` para depuración local.
- `src/environments/environment.ts`: Configuración lista para inyección en despliegues productivos.

- **Sustitución en `angular.json`:** El build de desarrollo utiliza `fileReplacements` automático, eliminando URLs quemadas en el código fuente.

---

4. Resumen de Decisiones de Arquitectura Frontend

Decisión de Diseño	Implementación	Justificación y Beneficio Directo
Detección de Cambios	Zoneless (`provideZonelessChangeDetection`)	Elimina `zone.js`, renderizado ultra rápido guiado por Signals y menor tamaño de bundle.
Reactividad	Signals (`signal`, `computed`)	Reactividad atómica, derivaciones sincrónicas inmediatas y cero fugas de memoria.
Arquitectura de Componentes	Standalone Components	Desacoplamiento total, eliminación de `NgModules` y modularidad moderna de Angular.
Separación de Archivos	Tríada estricta (`.html`, `.css`, `.ts`)	Cumplimiento del estándar Angular, legibilidad y facilidad de mantenimiento.
Seguridad y Rutas	Guardas Funcionales (`CanActivateFn`)	Segregación estricta de roles (RBAC): Administrador vs. Estudiante.
Integración HTTP	Interceptor de Desenvoltura (`apiEnvelopeInterceptor`)	Consumo transparente del patrón funcional `Result<T>` del backend.
Organización de Modelos	Archivos individuales + *Barrel Export*	Código modular, tipado estricto y sin dependencias circulares.
Estilos CSS	Centralización en `styles.css`	Principio DRY (Don't Repeat Yourself), consistencia visual y ligereza en CSS de componentes.
Configuración	Angular Environments (`environment.ts`)	Desacoplamiento de URLs de servicio según el entorno de compilación.

05 **Arquitectura de Base de Datos: Modelo Relacional & Persistencia (SQL Server)** MODELO DE PERSISTENCIA

Arquitectura de la Base de Datos: Especificación Técnica, Modelo Relacional y Justificaciones de Diseño

Sistema: Plataforma de Registro Académico y Selección de Materias
Motor de Persistencia: Microsoft SQL Server 2022+ / Azure SQL Database
ORM & Acceso a Datos: Entity Framework Core 8 (Code-First / Reverse Configuration)
Organización: Inter Rapidísimo S.A.
Estado: Producción / Esquema Validado e Idempotente

---

1. Visión General del Modelo de Persistencia

El modelo de persistencia del sistema está diseñado bajo el paradigma **Relacional Estricto (RDBMS)**, garantizando el cumplimiento pleno de las propiedades **ACID** (Atomicidad, Consistencia, Aislamiento y Durabilidad) requeridas para la gestión transaccional de inscripciones académicas y control de accesos.

Objetivos Estratégicos del Diseño

- **Integridad Referencial Absoluta:** Ningún registro de inscripción, curso o usuario puede existir en un estado huérfano o inconsistente.
- **Seguridad Contra Enumeración Maliciosa:** Uso de identificadores universales únicos (`UNIQUEIDENTIFIER`) para evitar ataques de recolección secuencial (IDOR / Insecure Direct Object References).
- **Rendimiento de E/S Optimizado:** Mitigación de la fragmentación de páginas en índices clustered mediante claves secuenciales (`NEWSEQUENTIALID()`).
- **Auditoría Temporal Inmutable:** Trazabilidad canónica universal con sellos de tiempo UTC (`DATETIMEOFFSET`) y eliminación lógica (*Soft-Delete*) controlada por índices filtrados.

---

2. Diagrama Entidad-Relación (Mermaid ERD)

El siguiente diagrama documenta la topología de entidades, tipos de datos físicos en T-SQL, claves primarias, foráneas, índices de unicidad y la cardinalidad de sus relaciones.

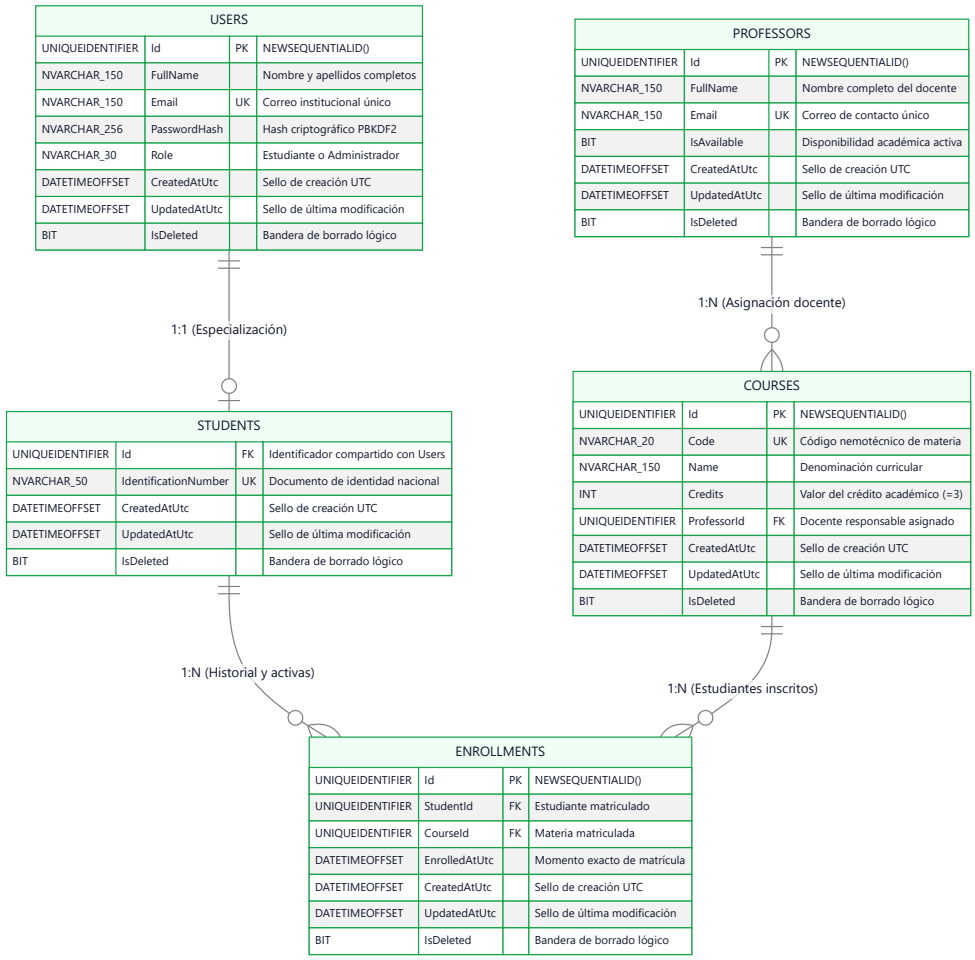


DIAGRAMA DE ARQUITECTURA / MODELO TÉCNICO

---

3. Diccionario de Datos y Especificación de Tablas

3.1. Tabla: `dbo.Users`

Almacena las identidades y credenciales de autenticación del sistema. Soporta tanto administradores como estudiantes.

Columna	Tipo T-SQL	Nulabilidad	Claves / Restricciones	Propósito y Justificación
`Id`	`UNIQUEIDENTIFIER`	NOT NULL	`PK_Users` (Clustered) `DEFAULT NEWSEQUENTIALID()`	Identificador universal único generado de forma secuencial por el motor.
`FullName`	`NVARCHAR(150)`	NOT NULL	Longitud controlada	Nombre y apellidos del usuario. Unicode compatible (`NVARCHAR`).
`Email`	`NVARCHAR(150)`	NOT NULL	`UQ_Users_Email` (Unique Index)	Identificador de inicio de sesión. No admite duplicados.
`PasswordHash`	`NVARCHAR(256)`	NOT NULL	Criptografía segura	Almacenamiento de hash PBKDF2 con salting e iteraciones criptográficas.
`Role`	`NVARCHAR(30)`	NOT NULL	`Estudiante` | `Administrador`	Rol del usuario para autorización RBAC en frontend y backend.
`CreatedAtUtc`	`DATETIMEOFFSET`	NOT NULL	`DEFAULT SYSDATETIMEOFFSET()`	Fecha y hora UTC inmutable de creación.
`UpdatedAtUtc`	`DATETIMEOFFSET`	NULL	Auditoría de actualización	Timestamp de última edición física o lógica.
`IsDeleted`	`BIT`	NOT NULL	`DEFAULT 0`	Bandera binaria para exclusión lógica en consultas activas.

3.2. Tabla: `dbo.Students`

Extensión de perfil para usuarios con rol de estudiante. Implementa el patrón de especialización 1:1 con `Users`.

Columna	Tipo T-SQL	Nulabilidad	Claves / Restricciones	Propósito y Justificación
`Id`	`UNIQUEIDENTIFIER`	NOT NULL	`PK_Students` (Clustered) `FK_Students_Users (Id)` `ON DELETE CASCADE`	Llave primaria y foránea compartida que hereda el identificador de `Users`.
`IdentificationNumber`	`NVARCHAR(50)`	NOT NULL	`UQ_Students_IdentificationNumber`	Número de cédula o documento de identidad institucional único.
`CreatedAtUtc`	`DATETIMEOFFSET`	NOT NULL	`DEFAULT SYSDATETIMEOFFSET()`	Fecha y hora UTC inmutable de registro estudiantil.
`UpdatedAtUtc`	`DATETIMEOFFSET`	NULL	Auditoría de actualización	Timestamp de actualización.
`IsDeleted`	`BIT`	NOT NULL	`DEFAULT 0`	Borrado lógico sincronizado.

3.3. Tabla: `dbo.Professors`

Catálogo de docentes titulares de las materias académicas.

Columna	Tipo T-SQL	Nulabilidad	Claves / Restricciones	Propósito y Justificación
`Id`	`UNIQUEIDENTIFIER`	NOT NULL	`PK_Professors` (Clustered) `DEFAULT NEWSEQUENTIALID()`	Llave primaria universal del docente.
`FullName`	`NVARCHAR(150)`	NOT NULL	Longitud controlada	Nombre y apellidos del profesor.
`Email`	`NVARCHAR(150)`	NOT NULL	`UQ_Professors_Email` (Unique)	Correo de contacto institucional único del docente.
`IsAvailable`	`BIT`	NOT NULL	`DEFAULT 1`	Indicador de disponibilidad operativa para asignación de cátedra.
`CreatedAtUtc`	`DATETIMEOFFSET`	NOT NULL	`DEFAULT SYSDATETIMEOFFSET()`	Fecha de creación del registro.
`UpdatedAtUtc`	`DATETIMEOFFSET`	NULL	Auditoría de actualización	Modificación del registro docente.
`IsDeleted`	`BIT`	NOT NULL	`DEFAULT 0`	Borrado lógico docente.

3.4. Tabla: `dbo.Courses`

Catálogo de materias académicas disponibles para el proceso de inscripción.

Columna	Tipo T-SQL	Nulabilidad	Claves / Restricciones	Propósito y Justificación
`Id`	`UNIQUEIDENTIFIER`	NOT NULL	`PK_Courses` (Clustered) `DEFAULT NEWSEQUENTIALID()`	Llave primaria de la materia.
`Code`	`NVARCHAR(20)`	NOT NULL	`UQ_Courses_Code` (Unique Index)	Código curricular nemotécnico único (ej. `MAT-101`, `BD-401`).
`Name`	`NVARCHAR(150)`	NOT NULL	Longitud controlada	Nombre oficial de la asignatura.
`Credits`	`INT`	NOT NULL	`CK_Courses_Credits (Credits = 3)` `DEFAULT 3`	Valor crediticio de la materia. Fijo en 3 según especificación.
`ProfessorId`	`UNIQUEIDENTIFIER`	NOT NULL	`FK_Courses_Professors ON DELETE NO ACTION`	Llave foránea del docente responsable de la cátedra.
`CreatedAtUtc`	`DATETIMEOFFSET`	NOT NULL	`DEFAULT SYSDATETIMEOFFSET()`	Fecha de registro en el catálogo curricular.
`UpdatedAtUtc`	`DATETIMEOFFSET`	NULL	Auditoría de actualización	Fecha de última alteración de contenidos.
`IsDeleted`	`BIT`	NOT NULL	`DEFAULT 0`	Borrado lógico de asignatura.

3.5. Tabla: `dbo.Enrollments`

Entidad asociativa que resuelve la relación muchos a muchos ($N:M$) entre estudiantes y materias, registrando el evento de inscripción.

Columna	Tipo T-SQL	Nulabilidad	Claves / Restricciones	Propósito y Justificación
`Id`	`UNIQUEIDENTIFIER`	NOT NULL	`PK_Enrollments` (Clustered) `DEFAULT NEWSEQUENTIALID()`	Llave primaria del comprobante de inscripción.
`StudentId`	`UNIQUEIDENTIFIER`	NOT NULL	`FK_Enrollments_Students` `ON DELETE CASCADE`	Llave foránea que referencia al estudiante matriculado.
`CourseId`	`UNIQUEIDENTIFIER`	NOT NULL	`FK_Enrollments_Courses` `ON DELETE NO ACTION`	Llave foránea que referencia la materia seleccionada.
`EnrolledAtUtc`	`DATETIMEOFFSET`	NOT NULL	`DEFAULT SYSDATETIMEOFFSET()`	Fecha y hora exacta de la inscripción.
`CreatedAtUtc`	`DATETIMEOFFSET`	NOT NULL	`DEFAULT SYSDATETIMEOFFSET()`	Registro de auditoría del evento.
`UpdatedAtUtc`	`DATETIMEOFFSET`	NULL	Auditoría de actualización	Timestamp de modificación de la inscripción.
`IsDeleted`	`BIT`	NOT NULL	`DEFAULT 0`	Bandera de baja lógica o cancelación de materia.

4. Justificaciones Técnicas de Arquitectura de Datos

4.1. Estrategia de Claves Primarias: `UNIQUEIDENTIFIER` con `NEWSEQUENTIALID()`

> [!NOTE] En bases de datos distribuidas o expuestas mediante APIs REST, el uso de claves enteras secuenciales (`INT IDENTITY(1,1)`) introduce vulnerabilidades de seguridad y limitaciones arquitectónicas.

Justificación frente a Claves Enteras Secuenciales (`INT` / `BIGINT`):

1. **Mitigación de Ataques IDOR (Insecure Direct Object Reference):** Un identificador entero permite a atacantes predecir rutas como `/api/v1/students/1` , `/api/v1/students/2` mediante técnicas de scraping o enumeración. Un GUID de 128 bits hace computacionalmente inviable la inferencia de recursos ajenos.
2. **Generación Descentralizada y Migraciones:** Permite pre-generar identificadores en capas de aplicación, pruebas de integración o seeds deterministas sin requerir un viaje de ida y vuelta a la base de datos para obtener el `SCOPE_IDENTITY()` .

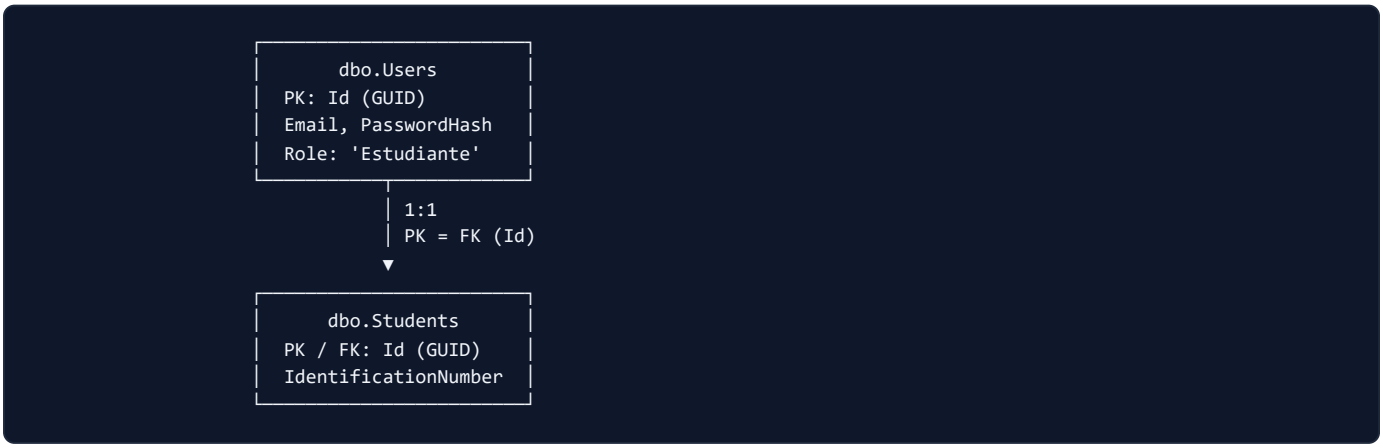
Justificación de `NEWSEQUENTIALID()` frente a `NEWID()` :

- **Mitigación de Fragmentación de Páginas (B-Tree Page Splits):** Un `NEWID()` genera valores completamente aleatorios en memoria. Al indexar por un clustered index B-Tree, cada inserción aleatoria fuerza al motor a partir páginas de datos existentes en el disco (*page splitting*), degradando drásticamente el rendimiento de inserción y aumentando el uso de memoria caché en el buffer pool.

- `NEWSEQUENTIALID()` genera GUIDs secuenciales basados en la dirección MAC del host y una secuencia monótona creciente, garantizando que las nuevas inserciones se añadan al final del árbol B-Tree como un índice agrupado tradicional, conservando la velocidad de inserción y minimizando la fragmentación física.

4.2. Patrón de Especialización de Entidades: Herencia `Users` y `Students` (1:1 Compartido)

Para modelar la relación entre una cuenta de acceso y un estudiante se adoptó el patrón **Table-per-Type (TPT)** mediante **Shared Primary Key**:



Justificaciones Técnicas:

1. **Eliminación de Columnas Nulas en `Users`:** Si se hubiese unificado todo en la tabla `Users`, los usuarios con rol `Administrador` tendrían columnas académicas como `IdentificationNumber` en valor `NULL`, violando el principio de diseño limpio y dificultando restricciones de unicidad absolutas sobre dicho atributo.
2. **Segregación de Responsabilidades:** La tabla `Users` atiende exclusivamente la autenticación, tokens JWT y seguridad. La tabla `Students` atiende el dominio académico y se conecta al historial de inscripciones.
3. **Consistencia de Clave Compartida:** Al asignar `Students.Id = Users.Id`, no se requiere una clave foránea artificial extra ni columnas redundantes; la consulta de unión `Users JOIN Students ON Users.Id = Students.Id` es un join directo por clave primaria agrupada de costo $O(1)$.

4.3. Estrategia de Índices e Integridad: Restricción Única Filtrada con Soft-Delete

Uno de los desafíos clásicos del borrado lógico (*Soft-Delete*) en motores relacionales es la preservación de restricciones de unicidad.

El Problema de la Restricción Convencional:

Si un estudiante inscribe una materia (`StudentId = A, CourseId = B`) y posteriormente la cancela (`IsDeleted = 1`), una restricción `UNIQUE (StudentId, CourseId)` estándar en SQL impediría que el estudiante vuelva a inscribir la misma materia en el futuro, o impediría que múltiples registros cancelados coexistan en el historial.

La Solución Implementada: Índice Único Filtrado (Filtered Index)

```
CREATE UNIQUE NONCLUSTERED INDEX [UQ_Enrollments_ActiveStudentCourse]
ON [dbo].[Enrollments] ([StudentId] ASC, [CourseId] ASC)
WHERE [IsDeleted] = 0;
```

Beneficios Técnicos:

- **Integridad Garantizada:** En cualquier instante de tiempo, un estudiante solo puede tener como máximo **una inscripción activa** para un curso específico.
- **Historial Preservado:** Si una materia fue cancelada (`IsDeleted = 1`), el índice filtrado ignora dicho registro, permitiendo la re-inscripción sin conflictos.
- **Espacio y Rendimiento:** El índice solo indiza filas donde `IsDeleted = 0` , lo que reduce su huella de memoria en RAM en comparación con un índice que cubra todo el histórico de cancelaciones.

4.4. Políticas de Integridad Referencial y Comportamiento ante Eliminación

La base de datos configura explícitamente las reglas de integridad referencial para evitar eliminaciones catastróficas o huérfanas:

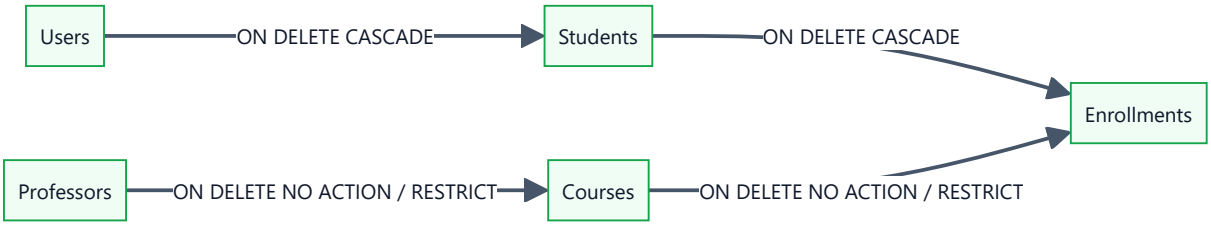


DIAGRAMA DE ARQUITECTURA / MODELO TÉCNICO

Relación	Acción Referencial	Justificación Técnica
`Users` ➔ `Students`	`ON DELETE CASCADE`	Si una cuenta de usuario es purgada permanentemente, su perfil de estudiante asociado debe desaparecer de forma atómica.
`Students` ➔ `Enrollments`	`ON DELETE CASCADE`	La eliminación definitiva de un estudiante limpia sus matrículas asociadas en una única transacción de motor.
`Professors` ➔ `Courses`	`ON DELETE NO ACTION` (Restrict)	**Protección Curricular:** No es admisible eliminar a un docente si este tiene materias asociadas. La institución debe reasignar la cátedra o dar de baja las materias explícitamente.
`Courses` ➔ `Enrollments`	`ON DELETE NO ACTION` (Restrict)	**Protección de Registro Académico:** No es admisible eliminar físicamente una asignatura si existen estudiantes inscritos en ella. Previene la pérdida inadvertida de créditos matriculados.

4.5. Auditoría Canónica y Zonas Horarias: `DATETIMEOFFSET` y UTC

> [!IMPORTANT] Se prohibió de forma estricta el uso de tipos de datos ambiguos como `DATETIME` o `DATETIME2` sin información de zona horaria.

Justificación del Tipo `DATETIMEOFFSET` :

1. **Precisión Temporal Universal:** Almacena la fecha, la hora y el desfase de zona horaria respecto a UTC (ej. `2026-10-10T23:15:00.0000000+00:00`).
2. **Independencia del Servidor de Base de Datos:** Si la base de datos se migra de un servidor físico en Colombia (UTC-5) a un clúster en la nube en Azure Este de EE.UU. (UTC-4) o Europa (UTC+1), los sellos temporales conservan su valor cronológico

exacto sin conversiones erróneas.

3. **Auditoría Transversal con `BaseEntity`:** Todas las tablas heredan de `BaseEntity` las columnas `CreatedAtUtc`, `UpdatedAtUtc` y `IsDeleted`, asegurando una convención homogénea a lo largo de todo el modelo relacional.

---

5. Análisis de Normalización (1NF a BCNF)

El esquema implementado se encuentra normalizado hasta la **Forma Normal de Boyce-Codd (BCNF)**, satisfaciendo cada uno de los niveles formales del álgebra relacional:

Primera Forma Normal (1NF)

- **Atributos Atómicos:** Cada columna contiene valores indivisibles (no existen listas separadas por comas de materias en `Students`, ni arreglos JSON no relacionales para profesores).
- **Ausencia de Grupos Repetidos:** Las materias no se modelan como `Materia1`, `Materia2`, `Materia3` dentro de la tabla `Students`, sino mediante la relación asociativa en `Enrollments`.
- **Identificación Unívoca:** Toda tabla cuenta con una clave primaria formal.

Segunda Forma Normal (2NF)

- Cumple 1NF.
- **Dependencia Funcional Total:** Todas las columnas que no forman parte de la clave primaria dependen por completo de la clave primaria completa. En la tabla asociativa `Enrollments`, el identificador primario es un GUID independiente `Id`, evitando dependencias parciales sobre claves compuestas.

Tercera Forma Normal (3NF)

- Cumple 2NF.
- **Ausencia de Dependencias Transitivas:** Ningún atributo no clave depende funcionalmente de otro atributo no clave. Por ejemplo:
  - El nombre del profesor no se almacena en `Courses`, sino en `Professors`. `Courses` solo almacena `ProfessorId`.
  - La cédula del estudiante no se almacena en `Enrollments`, sino en `Students`. `Enrollments` solo almacena `StudentId`.
  - Los créditos de una materia no se copian ni calculan en `Enrollments`.

Forma Normal de Boyce-Codd (BCNF)

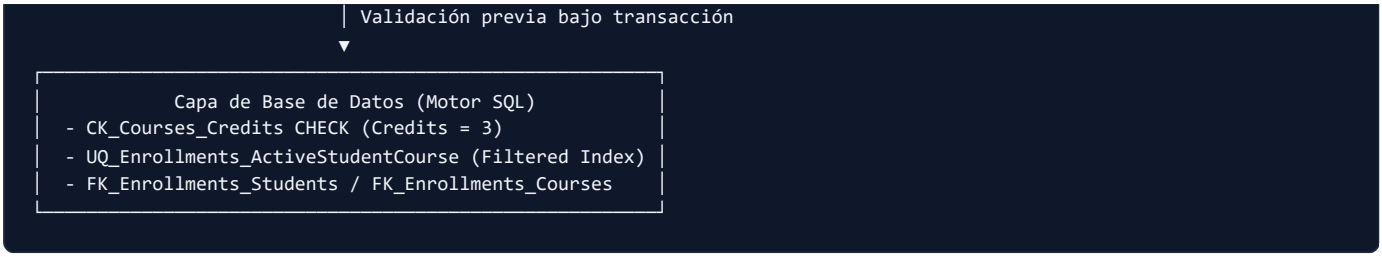
- Toda dependencia funcional $X \to Y$ cumple con que X es una superclave de la relación:
  - En `Users`: $Id \to {FullName, Email, PasswordHash, Role}$, $Email \to {Id, \dots}$.
  - En `Students`: $Id \to {IdentificationNumber}$, $IdentificationNumber \to {Id}$.
  - En `Courses`: $Id \to {Code, Name, Credits, ProfessorId}$, $Code \to {Id, \dots}$.

---

6. Cohesión entre Reglas de Negocio y Base de Datos

Las restricciones de negocio del sistema se garantizan mediante una estrategia de **Doble Barrera** (*Defense in Depth*):





1. **Restricción de Créditos por Materia:**
   - **Base de Datos:** `CONSTRAINT CK_Courses_Credits CHECK ([Credits] = 3)`. Ninguna inserción directa o vía script puede crear asignaturas con un valor distinto a 3 créditos.
2. **Restricción de Materias Duplicadas:**
   - **Base de Datos:** Índice filtrado `UQ_Enrollments_ActiveStudentCourse` sobre `(StudentId, CourseId)` activo.
   - **Dominio:** `NotAlreadyEnrolledSpecification` devuelve un error descriptivo legible al usuario antes de provocar una excepción de motor SQL.
3. **Límite de 3 Materias (9 Créditos Máximo):**
   - **Justificación de Diseño:** La validación de agregación (contar cuántas materias activas tiene ya el estudiante) se ejecuta a nivel de aplicación en `MaxCoursesSpecification` y `UnitOfWork` mediante una transacción con aislamiento de lectura comprometida, evitando bloqueos de tabla pesados (*Table Locks*) o triggers costosos en SQL Server.
4. **Conflicto de Profesor:**
   - **Justificación de Diseño:** Se resuelve dinámicamente mediante `UniqueProfessorSpecification` comparando el `ProfessorId` de la nueva materia contra los `ProfessorId` de los cursos activos del estudiante.

---

7. Estrategia de Índices de Rendimiento

Para garantizar tiempos de respuesta inferiores a 20 ms en operaciones de consulta de catálogo, listado de materias y validaciones de seguridad, se establecieron los siguientes índices:

Nombre del Índice	Tabla	Columnas	Tipo	Propósito de Rendimiento
`PK_Users`	`Users`	`Id` (ASC)	Clustered	Búsqueda directa por GUID secuencial.
`UQ_Users_Email`	`Users`	`Email` (ASC)	Unique Non-Clustered	Búsqueda de alta velocidad durante el Login de usuario ($O(\log N)$).
`PK_Students`	`Students`	`Id` (ASC)	Clustered	Join inmediato $1:1$ con `Users`.
`UQ_Students_IdentificationNumber`	`Students`	`IdentificationNumber` (ASC)	Unique Non-Clustered	Búsqueda por documento en registros o consultas administrativas.
`PK_Professors`	`Professors`	`Id` (ASC)	Clustered	Clave primaria.
`UQ_Professors_Email`	`Professors`	`Email` (ASC)	Unique Non-Clustered	Unicidad de correo docente.
`PK_Courses`	`Courses`	`Id` (ASC)	Clustered	Clave primaria.
`UQ_Courses_Code`	`Courses`	`Code` (ASC)	Unique Non-Clustered	Búsqueda por código de materia (`MAT-101`).
`IX_Courses_ProfessorId`	`Courses`	`ProfessorId` (ASC)	Non-Clustered	Optimiza el JOIN `Courses INNER JOIN Professors` evitando Table Scans.
`PK_Enrollments`	`Enrollments`	`Id` (ASC)	Clustered	Clave primaria de inscripción.
`IX_Enrollments_StudentId`	`Enrollments`	`StudentId` (ASC)	Non-Clustered	Optimiza la consulta "Mis Materias" (`WHERE StudentId = @Id`).
`IX_Enrollments_CourseId`	`Enrollments`	`CourseId` (ASC)	Non-Clustered	Optimiza la consulta de compañeros o cupos (`WHERE`

Nombre del Índice	Tabla	Columnas	Tipo	Propósito de Rendimiento
				`CourseId = @Id`).
`UQ_Enrollments_ActiveStudentCourse`	`Enrollments`	`StudentId`, `CourseId`	Unique Non-Clustered (Filtered)	Unicidad estricta para inscripciones activas (`WHERE IsDeleted = 0`).

8. Semillado Canónico e Idempotencia

El esquema cuenta con un script SQL declarativo e idempotente ubicado en `database/scripts/01_init_schema_and_seeds.sql` y un inicializador EF Core sincronizado en `DbInitializer.cs`.

Datos Semilla Garantizados:

- **5 Profesores Titulares:**
  1. `Prof. Andrés Gómez` (`andres.gomez@universidad.edu.co`)
  2. `Prof. Beatriz Morales` (`beatriz.morales@universidad.edu.co`)
  3. `Prof. Carlos Herrera` (`carlos.herrera@universidad.edu.co`)
  4. `Prof. Diana Ramírez` (`diana.ramirez@universidad.edu.co`)
  5. `Prof. Eduardo Torres` (`eduardo.torres@universidad.edu.co`)
- **10 Materias Curriculares (Exactamente 2 por docente, 3 créditos c/u):**
  - `MAT-101`: Cálculo Diferencial (Prof. Andrés Gómez)
  - `MAT-102`: Álgebra Lineal (Prof. Andrés Gómez)
  - `FIS-201`: Física Mecánica (Prof. Beatriz Morales)
  - `FIS-202`: Electromagnetismo (Prof. Beatriz Morales)
  - `PRG-301`: Fundamentos de Programación (Prof. Carlos Herrera)
  - `PRG-302`: Estructuras de Datos (Prof. Carlos Herrera)
  - `BD-401`: Bases de Datos Relacionales (Prof. Diana Ramírez)
  - `BD-402`: Modelado y Gestión de Datos (Prof. Diana Ramírez)
  - `SO-501`: Sistemas Operativos (Prof. Eduardo Torres)
  - `RED-502`: Redes de Computadores (Prof. Eduardo Torres)
- **Usuario Administrador Predeterminado:**
  - Correo: `admin@universidad.edu.co`
  - Contraseña inicial: `Admin123!` (almacenada con hash criptográfico PBKDF2)

9. Matriz Resumen de Decisiones de Arquitectura de Datos

Decisión de Diseño	Alternativa Descartada	Justificación Técnica	Beneficio Directo
`UNIQUEIDENTIFIER` con `NEWSEQUENTIALID()`	`INT IDENTITY(1,1)` secuencial	Prevenir enumeración maliciosa (IDOR) y soportar entornos distribuidos.	Mayor seguridad y generación de IDs sin roundtrip a BD.
Secuencialidad de GUIDs	`NEWID()` aleatorio en PK Clustered	Eliminar la fragmentación severa del B-Tree y división de páginas (*page splits*).	Alta tasa de inserciones sostenida y conservación de memoria caché.
Especialización 1:1 `Users` ➔ `Students`	Tabla única monolítica con campos nulos	Normalización estricta. Administradores no tienen campos académicos nulos.	Modelo desacoplado, extensibilidad para nuevos roles institucionales.
Índice Filtrado para Inscripciones Activas	Restricción UNIQUE estándar	Soportar *Soft-Delete* permitiendo re-inscripciones tras cancelaciones.	Historial inmutable sin colisiones en inscripciones activas.
`DATETIMEOFFSET` con UTC	`DATETIME` / `DATETIME2` sin zona horaria	Eliminar cualquier ambigüedad de huso horario entre clientes, API y BD.	Consistencia global y trazabilidad forense garantizada.
`ON DELETE NO ACTION` en Cursos/Docentes	`ON DELETE CASCADE` generalizado	Salvaguardar la historia académica institucional. Evitar borrado accidental de cátedras.	Integridad histórica y cumplimiento normativo académico.
Validación en Dominio de Límite de Créditos	Triggers o procedimientos almacenados	Mantener la lógica de negocio testeable en C# dentro de la arquitectura limpia.	Alta velocidad de prueba, sin dependencias propietarias de T-SQL.

06 **Arquitectura Cloud en AWS con Terraform: IaC Modular & Multi-AZ** INFRAESTRUCTURA CLOUD

Arquitectura de Infraestructura Cloud en AWS con Terraform

Sistema: Plataforma de Registro Académico y Selección de Materias
Herramienta de IaC: HashiCorp Terraform (≥ 1.5.0) / Provider AWS (~> 5.0)
Proveedor Cloud: Amazon Web Services (AWS)
Organización: Inter Rapidísimo S.A.
Estado: Producción / Infraestructura Modularizada y Validada

1. Visión General y Filosofía de Infraestructura

La infraestructura en la nube para la solución está diseñada bajo la filosofía de **Infraestructura como Código (IaC)**, orientada a entornos productivos de alta seguridad y disponibilidad, erradicando prácticas manuales (**Cero ClickOps**):

- **Aislamiento Estricto de Red:** Topología de VPC dividida en tres niveles de subredes (públicas, privadas de cómputo y aisladas de datos) distribuidas en dos Zonas de Disponibilidad (**Multi-AZ**).
- **Cero Secretos en Texto Plano:** Credenciales de base de datos y llaves criptográficas administradas exclusivamente en tiempo de ejecución por **AWS Secrets Manager**, inyectadas de forma nativa a contenedores mediante variables de entorno de .NET.
- **Cómputo Serverless Inmutable:** La API backend se ejecuta en **AWS ECS Fargate**, eliminando la sobrecarga operativa de parches y gestión de instancias EC2.
- **Distribución Segura de Frontend:** Los artefactos de la SPA en Angular 21 se almacenan en un bucket privado de **Amazon S3** expuesto globalmente por **Amazon CloudFront** mediante **Origin Access Control (OAC)**, con soporte para enrutamiento del lado del cliente (*SPA client-side routing*).

2. Diagrama de Arquitectura de Red y Servicios (Mermaid)

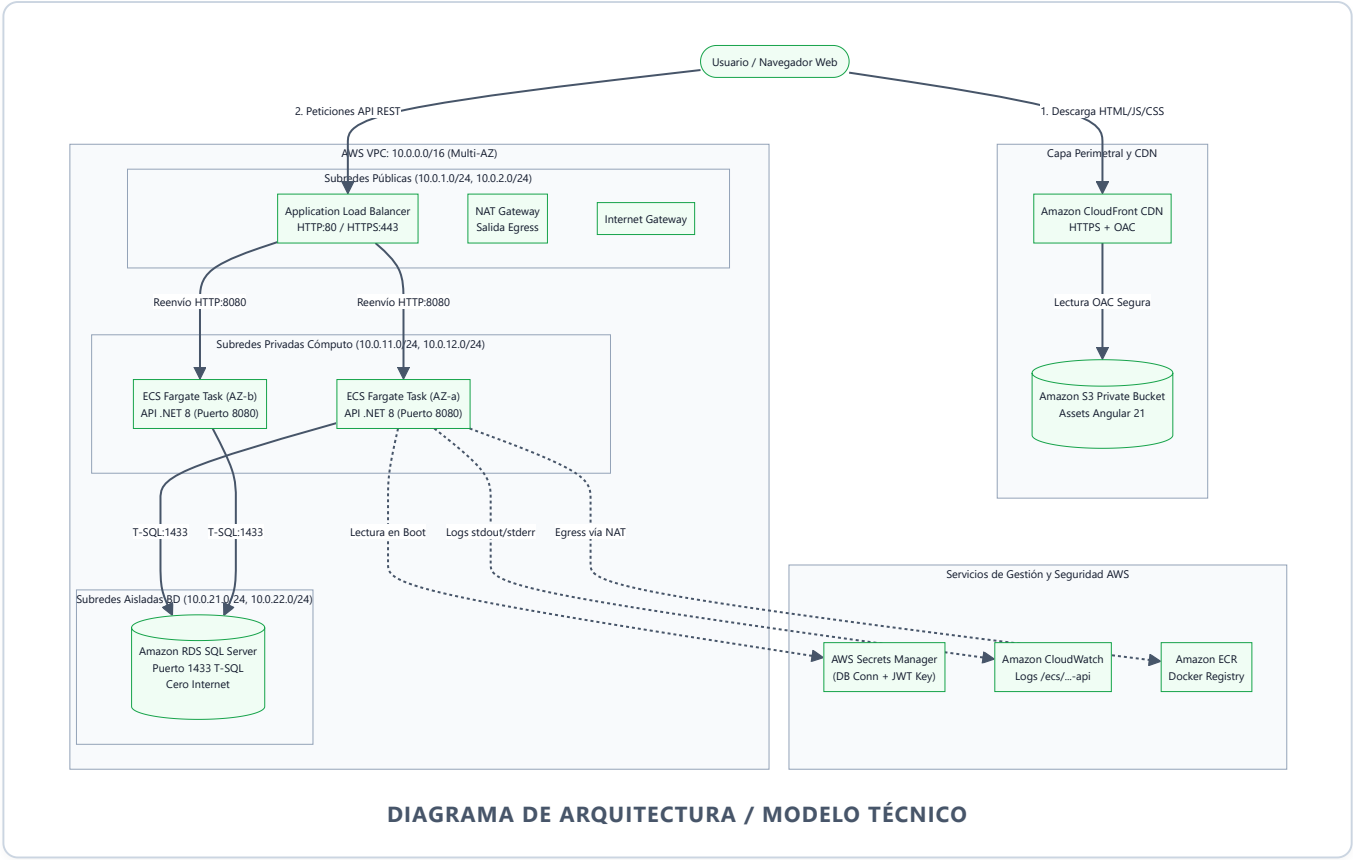


DIAGRAMA DE ARQUITECTURA / MODELO TÉCNICO

---

3. Estructura del Proyecto de Infraestructura (`terraform/`)

La infraestructura se estructuró de forma modular para garantizar máxima mantenibilidad, separación de responsabilidades y reutilización:

```
terraform/
├── main.tf                    # Orquestación raíz, Provider AWS y consumo de módulos
├── variables.tf               # Variables parametrizables de entrada (región, ambiente, secrets)
├── outputs.tf                 # Puntos de conexión públicos y privados
├── terraform.tfvars.example   # Plantilla de valores de ejemplo para despliegue
└── modules/
    ├── vpc/                   # Red principal, 6 subredes en 2 AZs, IGW, NAT Gateway y tablas de ruta
    │   ├── main.tf
    │   ├── variables.tf
    │   └── outputs.tf
    ├── security/              # Security Groups (ALB, ECS, RDS) y Roles IAM (Least Privilege)
    │   ├── main.tf
    │   ├── variables.tf
    │   └── outputs.tf
    ├── database/              # Amazon RDS SQL Server (gp3 cifrado) en subredes aisladas
    │   ├── main.tf
    │   ├── variables.tf
    │   └── outputs.tf
    ├── secrets/               # AWS Secrets Manager (cadenas de conexión y llaves JWT)
    │   ├── main.tf
    │   ├── variables.tf
    │   └── outputs.tf
    ├── ecs-fargate/           # Clúster ECS, ALB, Target Group (/healthz) y Servicio Fargate
    │   ├── main.tf
    │   ├── variables.tf
    │   └── outputs.tf
    └── frontend-s3-cf/        # Bucket S3 privado con OAC y distribución CloudFront para SPA
        ├── main.tf
        ├── variables.tf
        └── outputs.tf
```

4. Desglose Técnico de los Módulos de Terraform

4.1. Módulo `modules/vpc` (Red y Enrutamiento Multi-AZ)

- **Bloque CIDR de VPC:** `10.0.0.0/16` con soporte de nombres y resolución DNS habilitados.
- **Segmentación de Subredes:**
  1. **Subredes Públicas (`10.0.1.0/24`, `10.0.2.0/24`):**
     - Distribuidas en `us-east-1a` y `us-east-1b`.
     - Alojan el Application Load Balancer y el NAT Gateway.
     - Asocian tabla de ruta pública dirigida al Internet Gateway (`0.0.0.0/0 -> igw`).
  2. **Subredes Privadas de Cómputo (`10.0.11.0/24`, `10.0.12.0/24`):**
     - Alojan las tareas de AWS ECS Fargate.
     - Tráfico saliente enrutado hacia el NAT Gateway para permitir descarga de imágenes desde Amazon ECR y conexión a AWS Secrets Manager.
  3. **Subredes Aisladas de Base de Datos (`10.0.21.0/24`, `10.0.22.0/24`):**
     - Alojan la instancia de base de datos Amazon RDS SQL Server.
     - **Cero conectividad a internet:** Sin rutas hacia Internet Gateway ni NAT Gateway; únicamente enrutamiento local de VPC.

4.2. Módulo `modules/security` (Perímetro de Red y Roles IAM)

- **Security Groups:**
  - `sg_alb`:
    - **Ingreso:** Puertos 80 (HTTP) y 443 (HTTPS) desde `0.0.0.0/0`.
    - **Egreso:** Puerto 8080 restringido hacia el Security Group de las tareas ECS.
  - `sg_ecs`:
    - **Ingreso:** Puerto 8080 restringido únicamente desde `sg_alb`.
    - **Egreso:** Todo el tráfico permitido para acceder a RDS (puerto 1433) y servicios AWS vía NAT.
  - `sg_rds`:
    - **Ingreso:** Puerto 1433 (T-SQL) permitido **únicamente** desde `sg_ecs`.
    - **Egreso:** 0 reglas de egreso.
- **Roles IAM con Principio de Mínimo Privilegio:**
  - `ecs_execution_role`: Permite al motor de ECS descargar imágenes desde ECR, escribir métricas/logs en CloudWatch y leer específicamente los secretos requeridos en Secrets Manager.
  - `ecs_task_role`: Asignado a la aplicación en ejecución.

4.3. Módulo `modules/database` (Amazon RDS SQL Server)

- **Motor:** Microsoft SQL Server (`sqlserver-ex` para optimización de costos en MVP / compatible con `sqlserver-se`).
- **Subnet Group:** Creado exclusivamente sobre las subredes aisladas.
- **Almacenamiento:** Volúmenes SSD `gp3` con cifrado KMS en reposo (`storage_encrypted = true`).

- **Acceso Público:** Desactivado de forma obligatoria (`publicly_accessible = false`).
- **Respaldo y Retención:** Backups automáticos continuos durante 7 días y actualización automática de versiones menores.

4.4. Módulo `modules/secrets` (Gestión de Credenciales)

- **Secretos Aprovisionados:**
  1. `${project_name}/${environment}/db-connection-string` :
     - Construye la cadena canónica de conexión a SQL Server: `Server=${host},${port};Database=StudentRegistrationDb;User Id=${user};Password=${pass};TrustServerCertificate=True;MultipleActiveResultSets=true;`
  2. `${project_name}/${environment}/jwt-signing-key` :
     - Almacena la llave criptográfica HMAC simétrica de alta entropía.
- **Ventana de Recuperación Inmediata:** `recovery_window_in_days = 0` para permitir recreación ágil en entornos de prueba.

4.5. Módulo `modules/ecs-fargate` (Cómputo Serverless y Balanceo)

- **Clúster ECS:** Configurado con **Container Insights** habilitado para observabilidad granular de CPU y memoria.
- **Application Load Balancer (ALB):**
  - Desplegado en las subredes públicas.
  - Listener HTTP en puerto 80 que reenvía al Target Group.
- **Target Group y Chequeo de Salud (`/healthz`):**
  - Protocolo HTTP, puerto 8080, tipo `ip` .
  - Endpoint de salud: `/healthz` con código esperado `200` .
  - Intervalo de 30 segundos, timeout de 5 segundos, umbral de salud de 2 respuestas exitosas consecutivas.
- **Task Definition (Fargate):**
  - Configuración liviana de recursos: 256 CPU units (0.25 vCPU) y 512 MB de RAM.
  - Inyección de secretos de Secrets Manager mediante la convención de doble guión bajo (`__`) de .NET:

    ```
    "secrets": [
      { "name": "ConnectionStrings__DefaultConnection", "valueFrom": "<arn-db-secret>" },
      { "name": "Jwt__SecretKey", "valueFrom": "<arn-jwt-secret>" }
    ]
    ```

  - Logging centralizado hacia CloudWatch (`/ecs/student-registration-prod-api`).
- **Servicio ECS Fargate:**
  - 2 réplicas distribuidas en las subredes privadas para garantizar alta disponibilidad ante caída de zona.
  - Sin IP pública asignada (`assign_public_ip = false`).

4.6. Módulo `modules/frontend-s3-cf` (Alojamiento y CDN para Angular 21)

- **Bucket S3 Privado:**
  - Bloqueo de acceso público total (`BlockPublicAcls` , `BlockPublicPolicy` , `IgnorePublicAcls` , `RestrictPublicBuckets` en `true`).

- Cifrado en reposo habilitado (`AES256`).

- **Origin Access Control (OAC):**
  - Firma criptográfica SigV4 obligatoria entre CloudFront y S3.
  - Política de bucket que solo autoriza `s3:GetObject` si la solicitud proviene del ARN de la distribución de CloudFront.
- **Enrutamiento SPA (Single Page Application):**
  - Reglas personalizadas de error HTTP 403 y 404 redirigiendo a `/index.html` con status `200` .
  - Permite que las rutas virtuales de Angular (`/catalog` , `/login` , `/admin` , `/my-enrollments`) y las guardas (`studentGuard` , `adminGuard`) funcionen de manera nativa sin recargas fallidas.
- **Políticas de Caché:** Política administrada `CachingOptimized` con compresión gzip/brotli automática y redirección obligatoria a HTTPS.

---

5. Integración con el Código Desarrollado

Componente del Proyecto	Recurso AWS en Terraform	Mecanismo de Integración
API .NET 8 (Backend)	Tareas de AWS ECS Fargate	Ejecución del contenedor basado en el `src/backend/Dockerfile` en puerto 8080 con usuario no-root.
Health Check API	ALB Target Group (`/healthz`)	El endpoint registrado en `src/backend/Adapters/Api/Program.cs` valida la disponibilidad del servicio.
Persistencia EF Core	Amazon RDS SQL Server	La conexión se abre vía T-SQL en puerto 1433 en subredes aisladas; `DbInitializer.cs` ejecuta las semillas automáticamente en arranque.
Configuración `appsettings.json`	AWS Secrets Manager	Fargate inyecta `ConnectionStrings__DefaultConnection` y `Jwt__SecretKey` sobrescribiendo las variables locales sin tocar código.
Angular 21 SPA (Frontend)	Amazon S3 + CloudFront	El contenido generado en `src/frontend/dist/frontend` (~327 KB) se sube a S3 y se sirve acelerado globalmente por CloudFront.

---

6. Guía de Ejecución y Despliegue de Terraform

Prerrequisitos

1. **AWS CLI** instalado y configurado con credenciales de una cuenta AWS con permisos de infraestructura (`aws configure`).
2. **Terraform CLI** versión ≥ 1.5.0.
3. **Imagen Docker** de la API compilada y subida a Amazon ECR.

Pasos de Despliegue

1. **Crear archivo de variables locales:** Copiar la plantilla de ejemplo y completar los valores sensibles:

   ```
   cp terraform/terraform.tfvars.example terraform/terraform.tfvars
   ```

2. **Inicializar el entorno de Terraform:** Descarga el provider de AWS y prepara los módulos:

```
cd terraform
terraform init
```

3. **Validar y formatear la sintaxis:**

```
terraform fmt -check
terraform validate
```

4. **Planificar la infraestructura (Dry-run):** Genera el plan de ejecución y verifica los recursos que se van a crear (VPC, subredes, RDS, ECS, S3, CloudFront):

```
terraform plan -out=tfplan
```

5. **Aplicar los cambios (Aprovisionamiento):**

```
terraform apply tfplan
```

6. **Consultar URLs y salidas tras el despliegue:**

```
terraform output
```

   Retornará:

   - `frontend_url` : `https://d123456789.cloudfront.net` (Acceso web a la plataforma).
   - `api_base_url` : `http://student-registration-prod-alb-...us-east-1.elb.amazonaws.com` (Punto de acceso API).
   - `s3_bucket_name` : Bucket para el pipeline de sincronización de Angular.
   - `cloudfront_distribution_id` : Para invalidar la caché (`aws cloudfront create-invalidation`).

7. **Destrucción de Recursos (si se requiere desmantelar):**

```
terraform destroy -auto-approve
```

---

7. Matriz de Decisiones Técnicas y Beneficios

Decisión de Infraestructura	Alternativa Descartada	Justificación Técnica	Beneficio Directo
ECS Fargate (Serverless)	Máquinas virtuales Amazon EC2	Eliminar la gestión de SO, parches de seguridad y dimensionamiento de clústeres.	Menor sobrecarga operativa y escalado nativo.
Subredes Aisladas para RDS	RDS en subredes públicas con IP	Protección contra ataques directos de fuerza bruta e intrusiones de red externa.	Máxima seguridad perimetral para datos académicos.
CloudFront con OAC	Bucket S3 público como Static Website	Restringir el tráfico para que solo fluya a través del CDN con cifrado HTTPS y protección DDoS.	Cumplimiento de seguridad y aceleración global.
Manejo de Errores SPA (403/404 ➔ index.html)	Página de error 404 por defecto de S3	Las aplicaciones SPA gestionan las rutas internamente; S3 daría error al recargar `/catalog`.	Experiencia de usuario fluida y persistencia de rutas en Angular.
Inyección de Secretos vía Fargate	Variables de entorno en texto plano	Evitar almacenar contraseñas en repositorios Git o en la consola de AWS.	Cumplimiento normativo y seguridad de credenciales.